

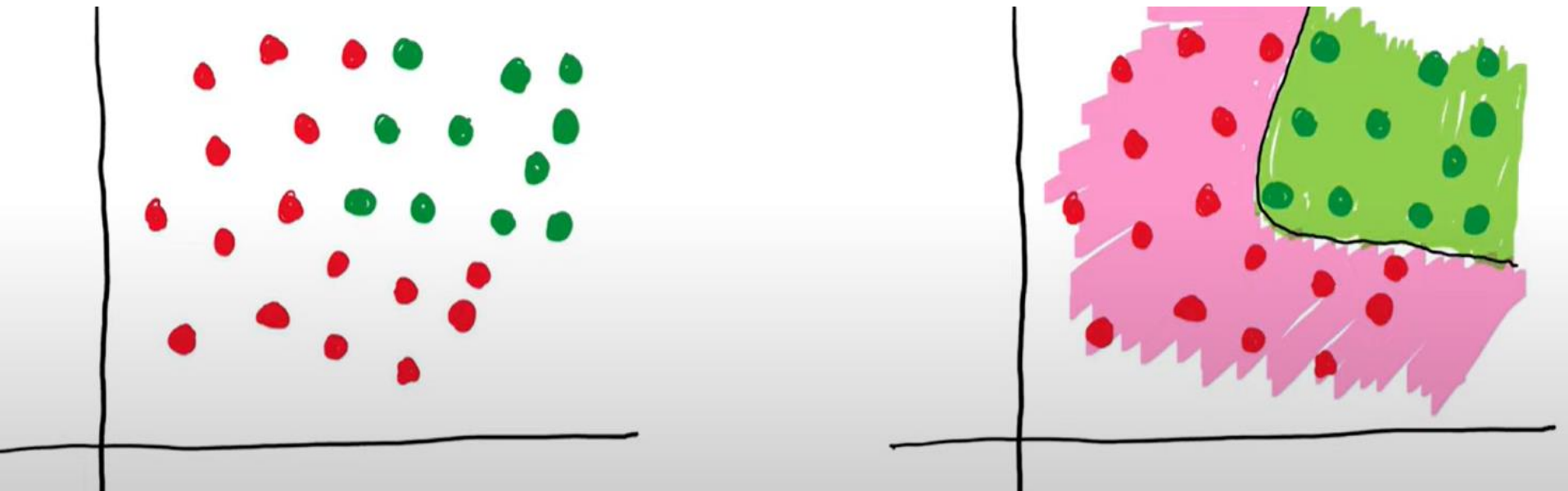
UNIT II

- Multi-layer Perceptron(Book-1) – Going Forwards – Going Backwards: Back Propagation Error – Multi-layer Perceptron in Practice – Examples of using the MLP – Overview - Deriving Back-Propagation – Radial Basis Functions and Splines – Concepts – RBF Network – Interpolations and Basis Functions – Support Vector Machines

The Multi-layer Perceptron

- Linear models are easy to understand and use, they come with the inherent cost that is implied by the word ‘linear’;
- They can only identify straight lines, planes, or hyperplanes.
- Majority of interesting problems are not linearly separable.
- Problems can be made linearly separable if we can work out how to transform the features suitably the learning in the neural network happens in the weights.
- To perform more computation it seems sensible to add more weights.
- There are two things that we can do: add some backwards connections, so that the output neurons connect to the inputs again, or add more neurons.

- If your data is non-linear, then single perceptron is not able to handle this. like below example (the data set with two different classes).

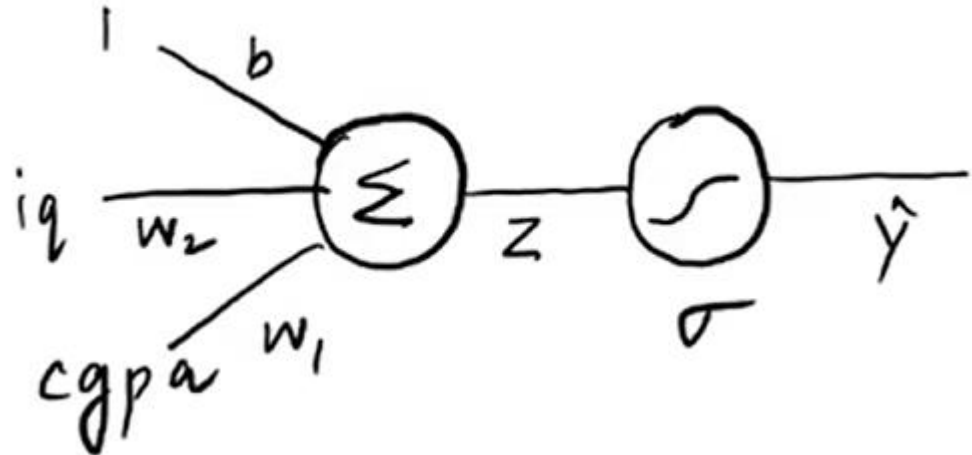


- To handle above non-linear data , we have given a boundary which is curve not a line.

- We have to design an algorithm, which should handle non-linear data by using perceptron only. so we make use of multi layer perceptron to handle non-linear data.

Perceptron with sigmoid:

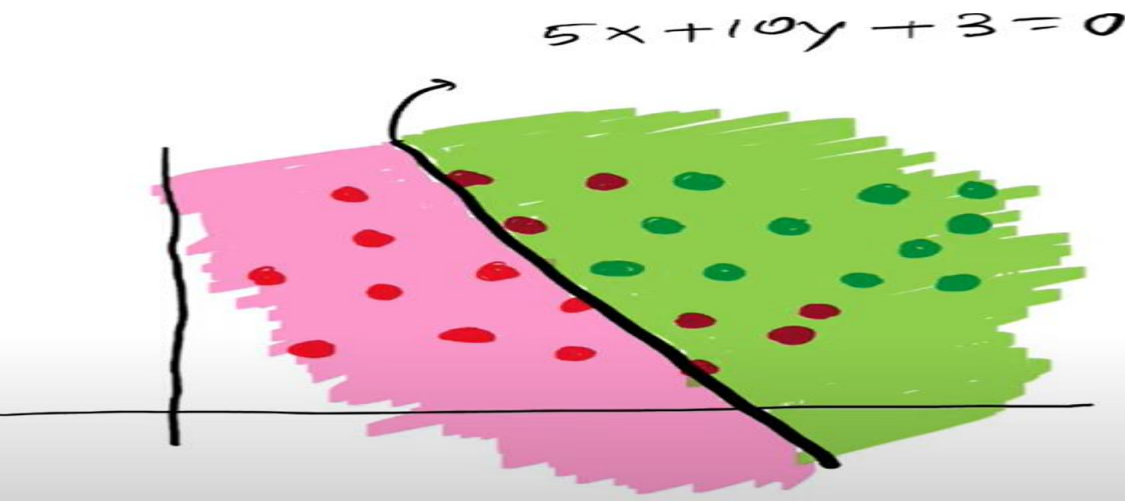
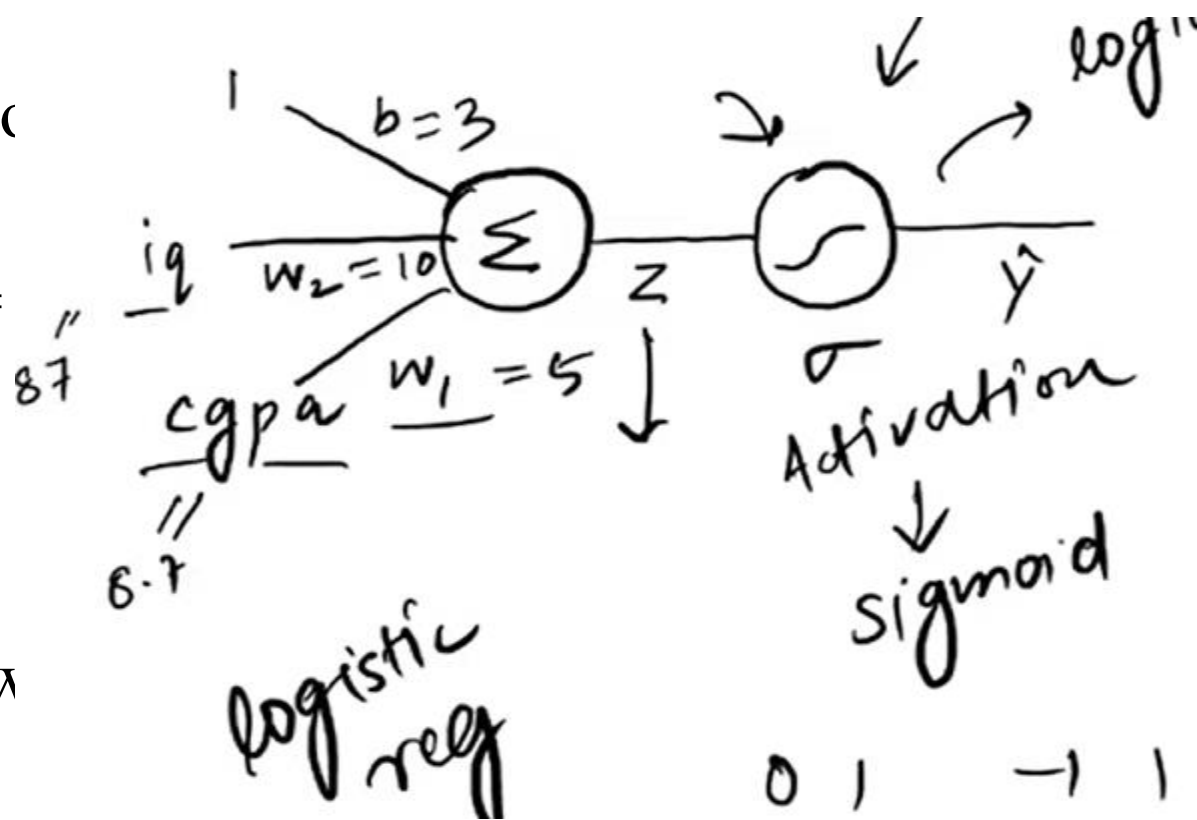
- In this perceptron, activation function is “sigmoid” function
- Two inputs iq and cgpa, output to predict “ placement or no placement”
- sigmoid output is 0 or 1 , -1 or 1
- We get probability in between 0 -1 to decide whether student can have placement or not.



- For example, when you train this model, we get student iq=87 and cgpa= weights $w_1=5$. $w_2=10$ $b=3$

- For the given values, we get the line like below with the equation:

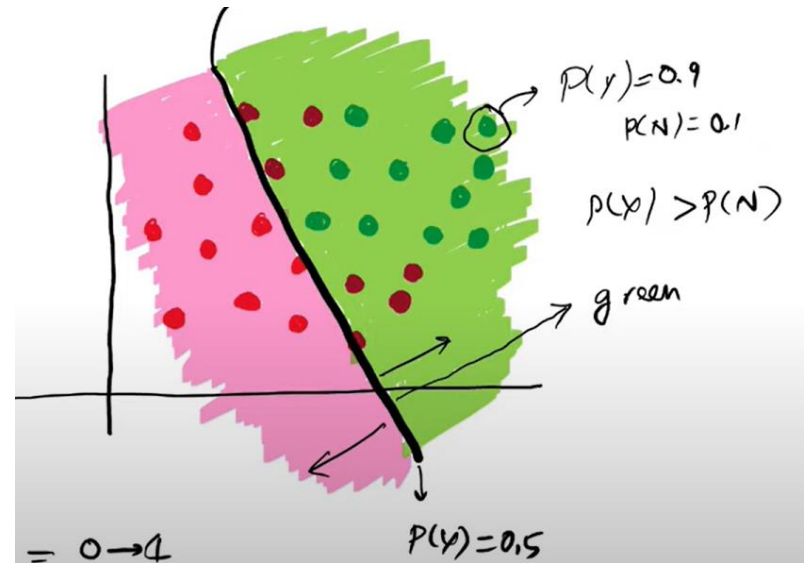
$$5x+10y+3=0$$



- Then calculate z.
- then send z value to sigmoid function

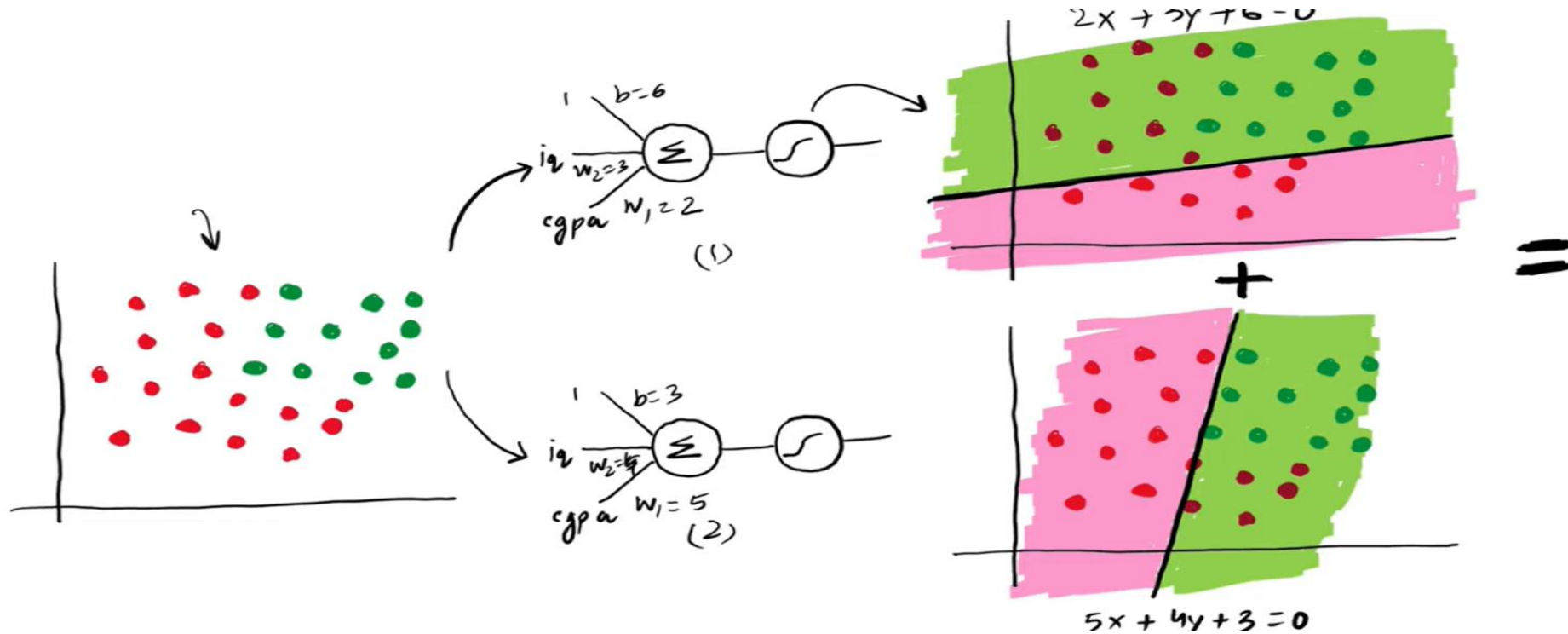
$$\begin{array}{l}
 w_1 \text{ cgpa} + w_2 \text{ iq} + b \\
 \downarrow \\
 5 \times 8.7 + 10 \times 87 + 3 = \textcircled{2} \rightarrow \frac{1}{1+e^{-z}} = 0 \rightarrow 4 \\
 P(Y) = 0.8
 \end{array}$$

- After calculating sigmoid, suppose you got probability as 0.8. it indicate the chance of placement is 0.8. so sigmoid will give you two probabilities, probability of yes (placement) and probability of no (no placement)



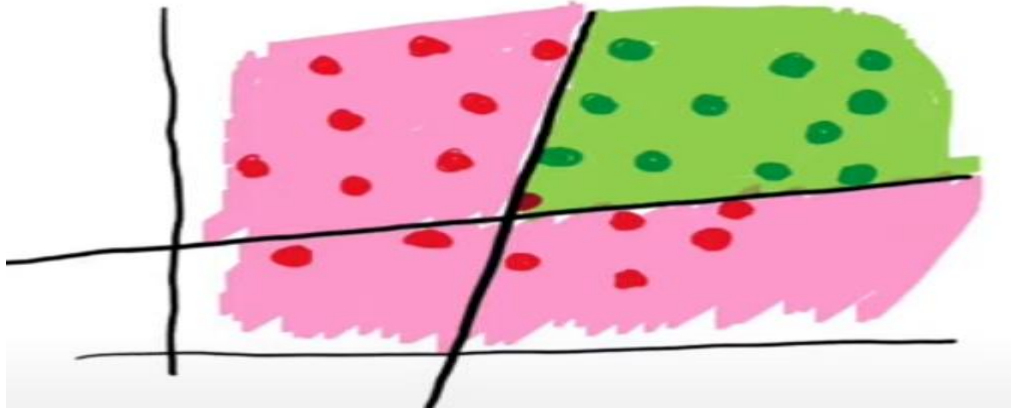
Multilayer perceptron:

- We have same non-linear data as above and trained two perceptron with some values.

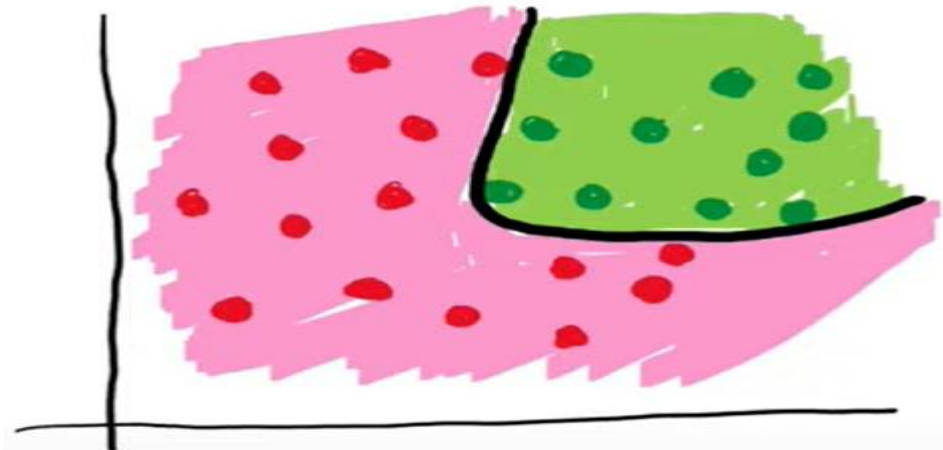


- The output (decision boundary) of these two perceptron will combine together

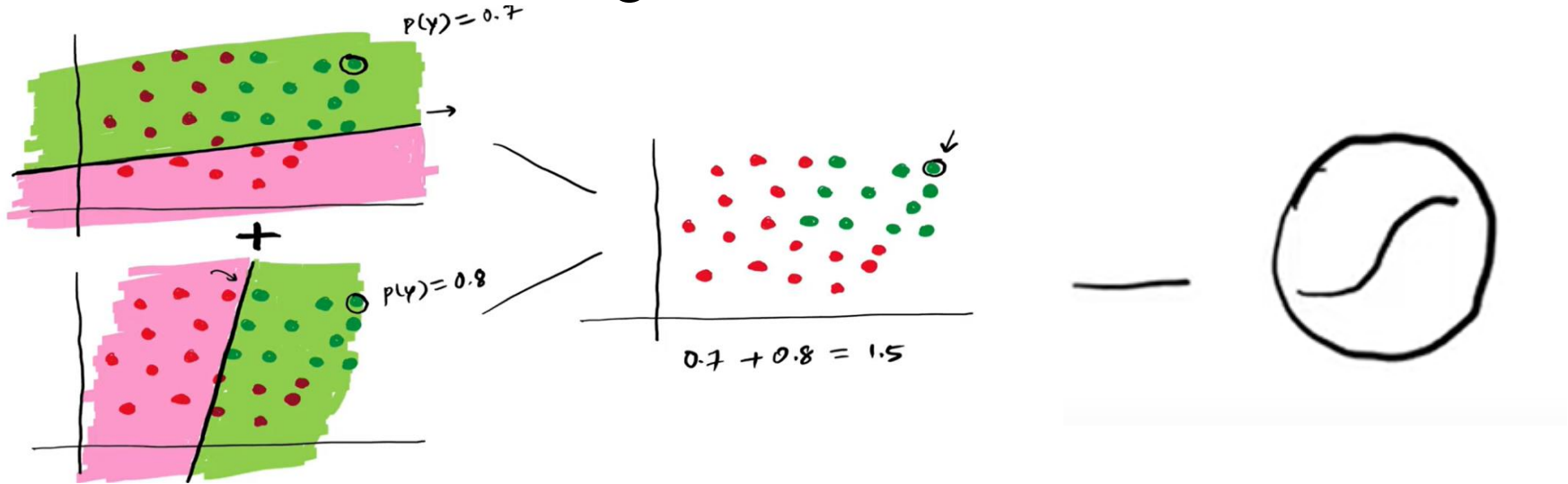
- The output (decision boundary) of these two perceptron will combine together, we get like below



- When we do some smoothing, we get like below.



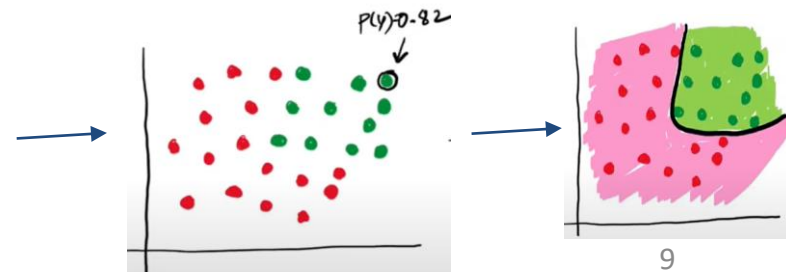
- Suppose we took one student probability in one perceptron as 0.7 and another perceptron has 0.8, when you combine the student value we got 1.5



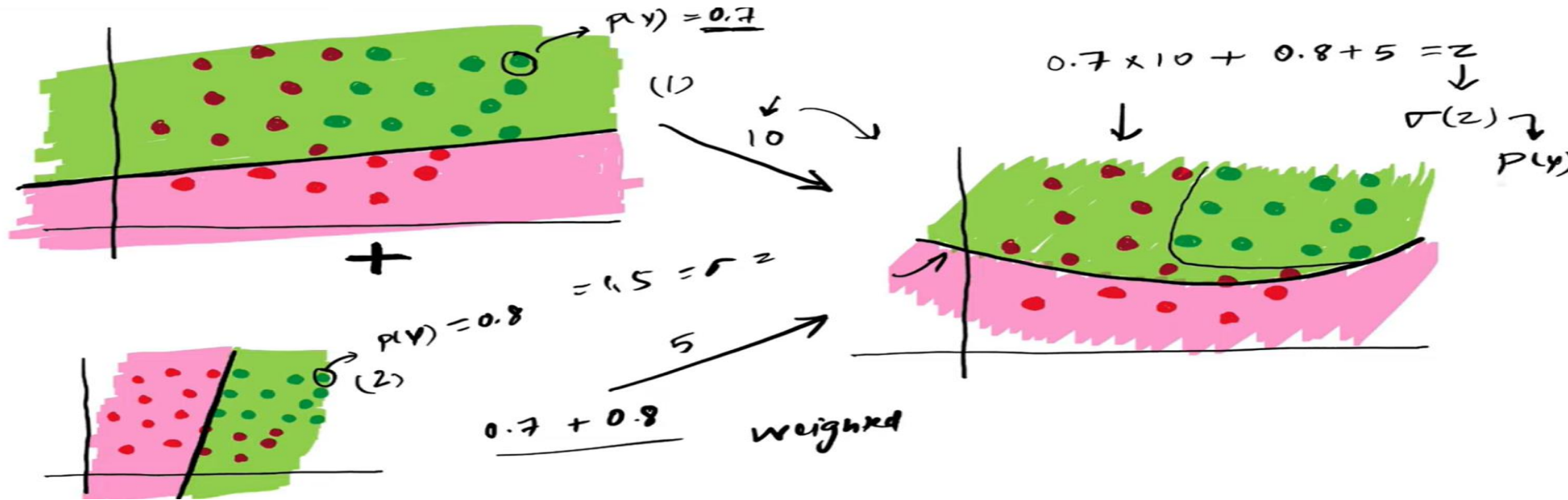
- The probability that we got is 1.5 but probability lies between 0-1 so I will give this value to sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

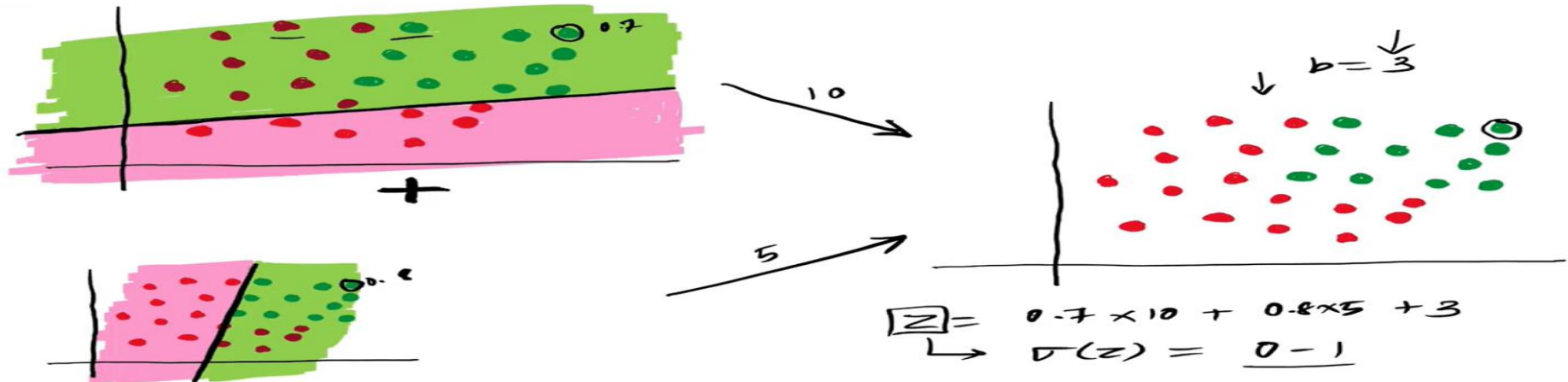
$$\frac{1}{1 + e^{-1.5}} = 0.82$$

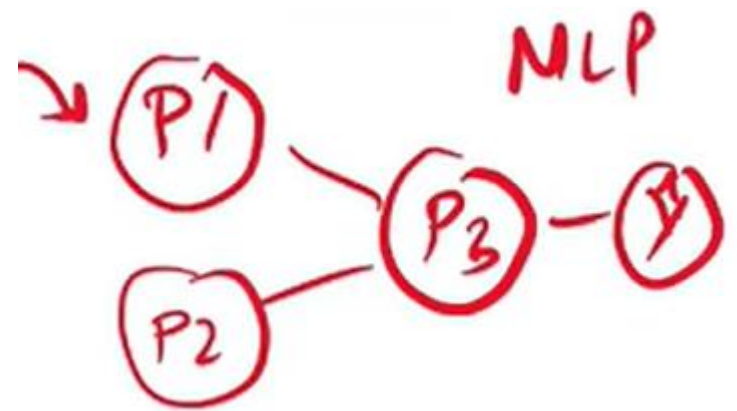
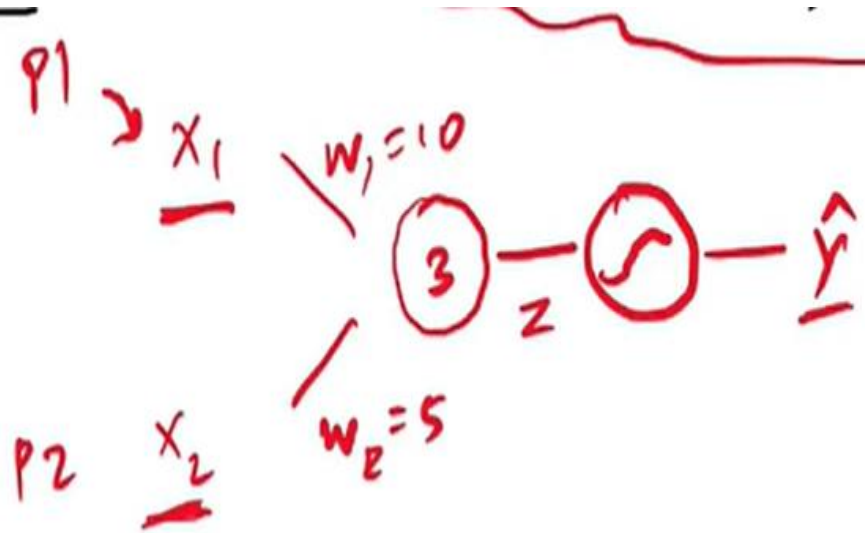


- Weighted addition:

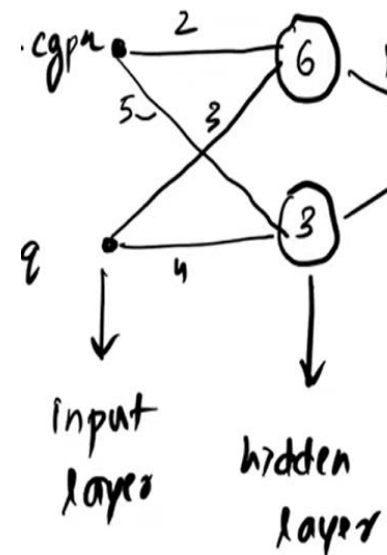
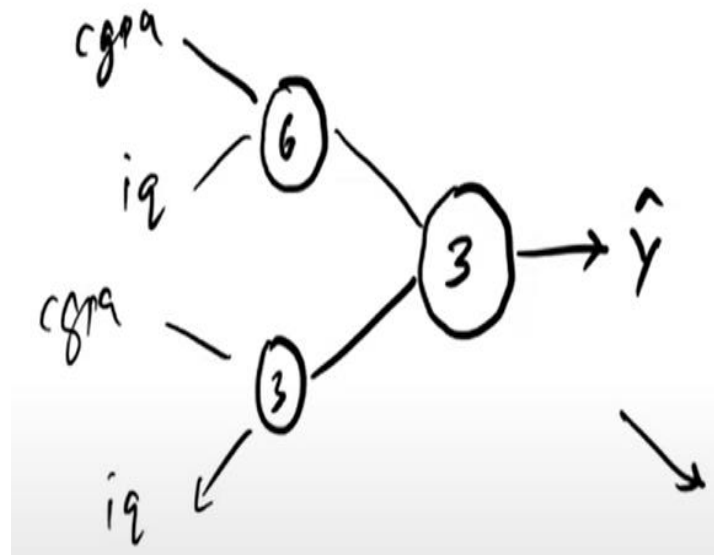
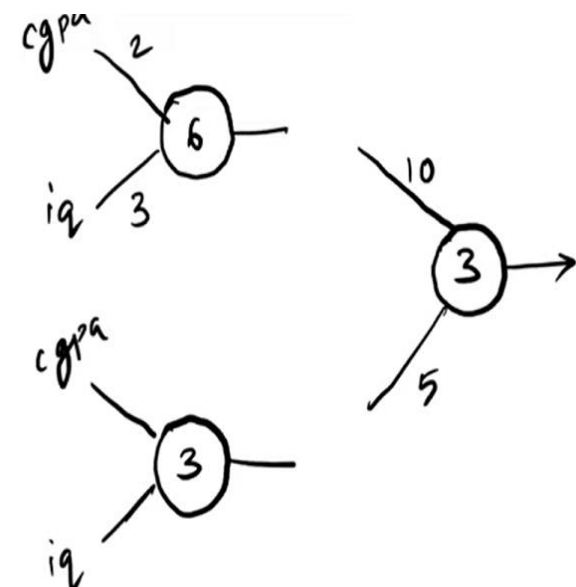


- We add bias also

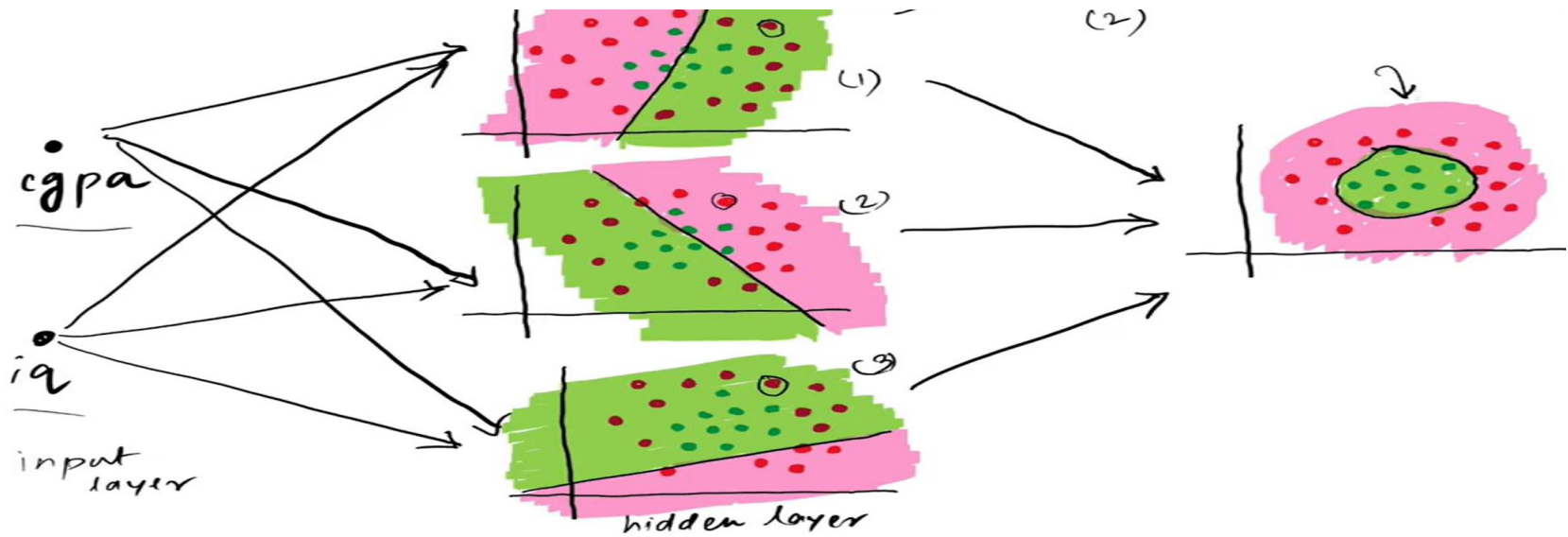




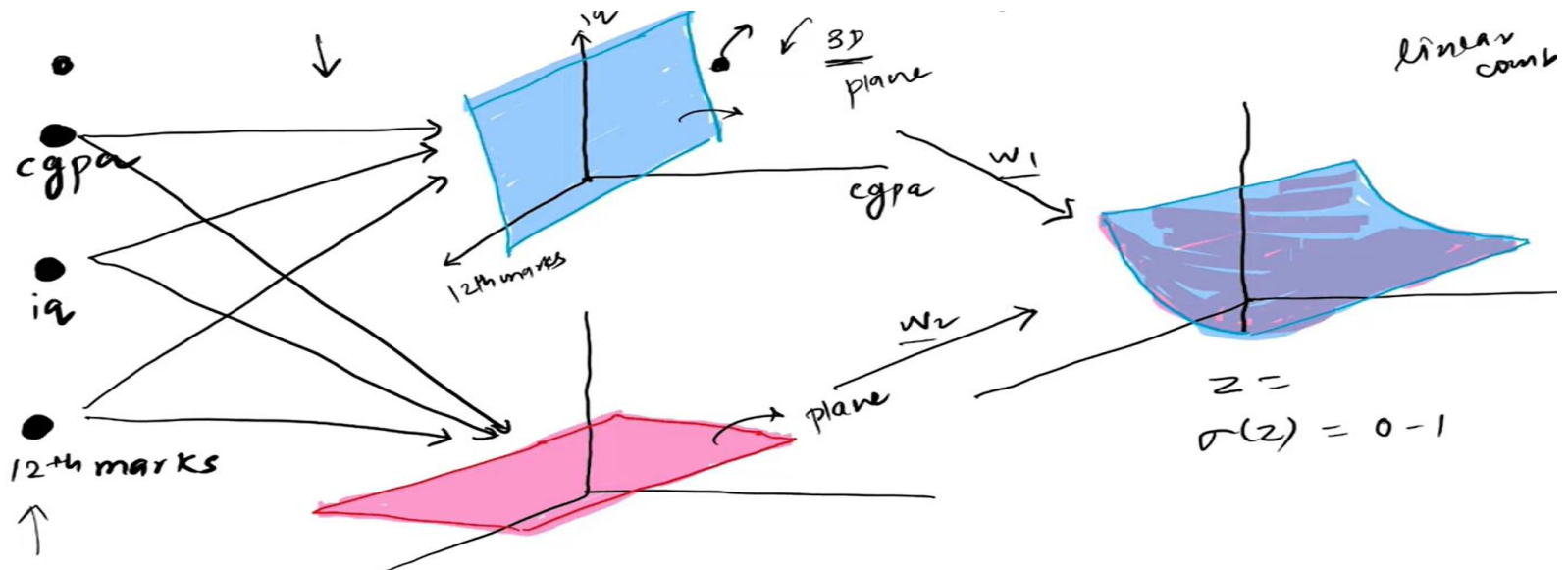
- We can draw the network as below:



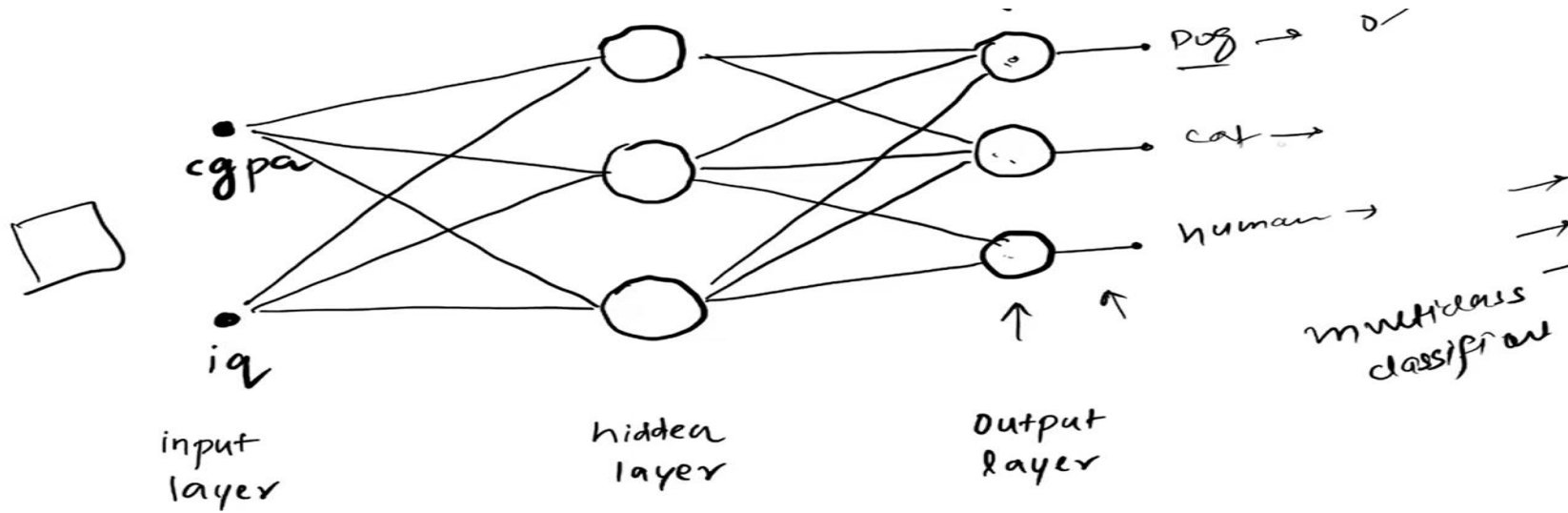
- Adding nodes in the hidden layers



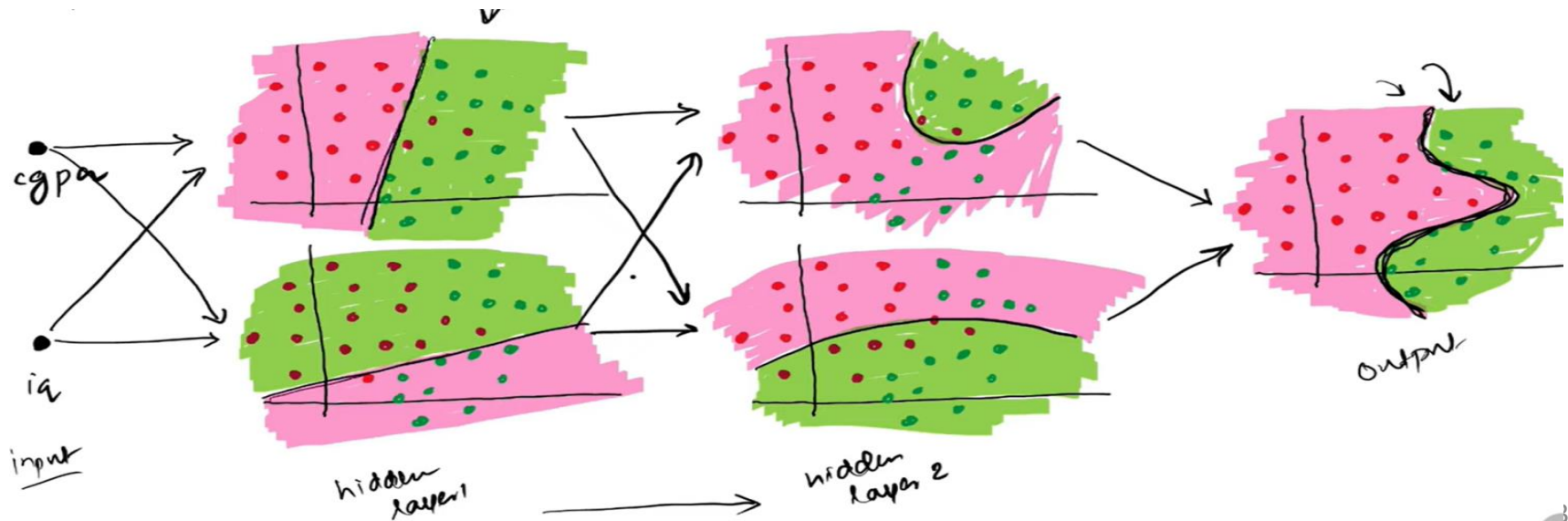
- Adding nodes in the input layers



- Adding nodes in the output layers



- Adding more hidden layers



- The first approach leads into recurrent networks.
- These have been studied, but are not that commonly used.
- Instead consider the second approach, add neurons between the input nodes and the outputs, and this will make more complex neural networks.
- Why adding extra layers of nodes makes a neural network more powerful.
- A prepared network can solve the two-dimensional XOR problem, something that we have seen is not possible for a linear model like the Perceptron.

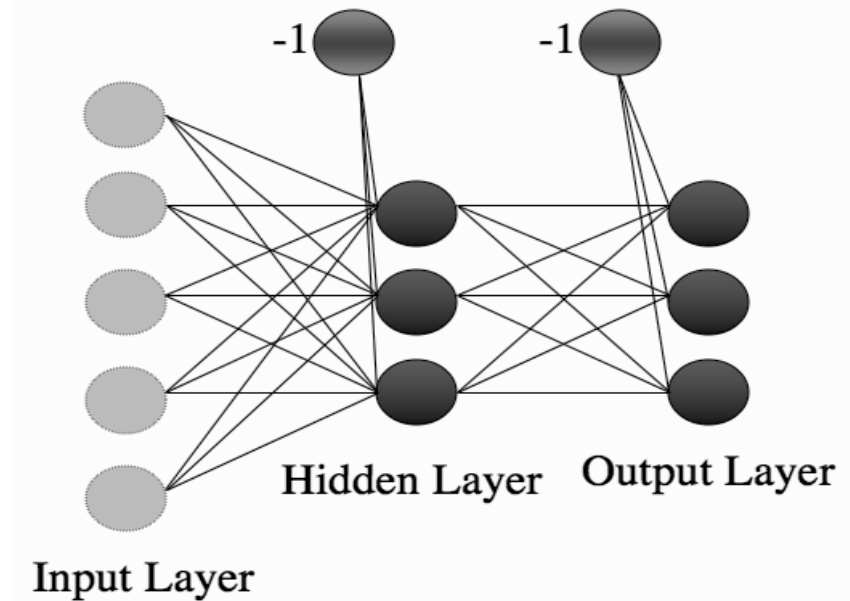
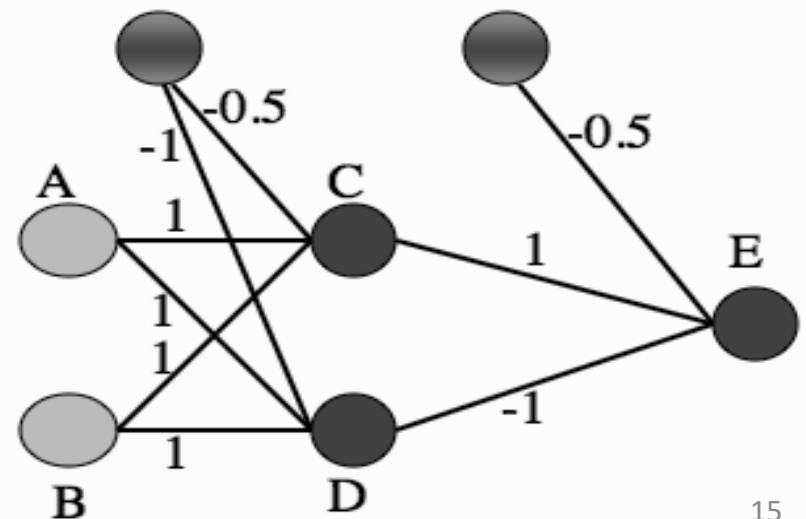


FIGURE 4.1 The Multi-layer Perceptron network, consisting of multiple layers of connected neurons.

- A suitable network is shown in Figure 4.2.
- To check that it gives the correct answers, all that is required is to put in each input and work through the network, treating it as two different Perceptrons, first computing the activations of the neurons in the middle layer (labelled as C and D in Figure 4.2) and then using those activations as the inputs to the single neuron at the output.
- As an example, I'll work out what happens when you put in $(1, 0)$ as an input; the job of checking the rest is up to you.

FIGURE 4.2 A Multi-layer Perceptron network showing a set of weights that solve the XOR problem.



- Input (1, 0) corresponds to node A being 1 and B being 0.
- The input to neuron C is therefore

$$-1 \times 0.5 + 1 \times 1 + 0 \times 1 = -0.5 + 1 = 0.5$$
- This is above the threshold of 0, and so neuron C fires, giving output 1. For neuron D the input is

$$-1 \times 1 + 1 \times 1 + 0 \times 1 = -1 + 1 = 0$$

 and so it does not fire, giving output 0.
- Therefore the input to neuron E is

$$-1 \times 0.5 + 1 \times 1 + 0 \times -1 = 0.5 \quad \text{so neuron E fires.}$$
- Checking the result of the inputs should persuade you that neuron E fires when inputs A and B are different to each other, but does not fire when they are the same, which is exactly the XOR function (it doesn't matter that the fire and not fire have been reversed).
- This network can solve a problem that the Perceptron cannot, it seems worth looking into further

- The MLP is one of the most common neural networks in use.
- It is often treated as a 'black box', in that people use it without understanding how it works, which often results in fairly poor results.

GOING FORWARDS

- Training the MLP consists of two parts: working out what the outputs are for the given inputs and the current weights, and then updating the weights according to the error, which is a function of the difference between the outputs and the targets.
- These are generally known as going forwards and backwards through the network.
- How to go forwards for the MLP is the XOR example

- It is same as the Perceptron, except that we have to do it twice, once for each set of neurons, and need to do it layer by layer, because otherwise the input values to the second layer don't exist.
- Made an MLP with two layers of nodes, there is no reason why we can't make one with 3, or 4, or 20 layers of nodes.
- Work forwards through the network computing the activations of one layer of neurons and using those as the inputs for the next layer.
- Start at the left by filling in the values for the inputs.
- Use these inputs and the first level of weights to calculate the activations of the hidden layer, and then use those activations and the next set of weights to calculate the activations of the output layer.
- Now got the outputs of the network, and compare them to the targets and compute the error.

BIASES

- Include a bias input to each neuron.
- Do this in the same way as for the Perceptron by having an extra input that is permanently set to -1, and adjusting the weights to each neuron as part of the training.
- Each neuron in the network (whether it is a hidden layer or the output) has 1 extra input, with fixed value.

GOING BACKWARDS: BACK-PROPAGATION OF ERROR

- Computing the errors at the output is no more difficult than it was for the Perceptron, but working out what to do with those errors is more difficult.
- The method that going to look at is called back-propagation of error, which makes it clear that the errors are sent backwards through the network.
- It is a form of gradient descent.

- The best way to describe back-propagation properly is mathematically, but this can be intimidating and difficult to get a handle on at first.
- There are actually just three things that need to know, all of which are from differential calculus:
 - The derivative of $\frac{1}{2} X^2$,
 - *If you differentiate a function of x with respect to some other variable t , then the answer is 0,*
 - *The chain rule, tells how to differentiate composite functions.*
- Talked about the Perceptron, changed the weights so that the neurons fired when the targets said they should, and didn't fire when the targets said they shouldn't.
- What we did was to choose an error function for each neuron k : $E_k = y_k - t_k$, and tried to make it as small as possible.
- Since there was only one set of weights in the network, this was sufficient to train the network.

- Minimise the error, so that neurons fire only when they should—but, with the addition of extra layers of weights, this is harder to arrange.
- The problem is that when we try to adapt the weights of the Multi-layer Perceptron, we have to work out which weights caused the error.
- This could be the weights connecting the inputs to the hidden layer, or the weights connecting the hidden layer to the output layer.
- The error function used for the Perceptron was $\sum_{k=1}^N E_k = \sum_{k=1}^N y_k - t_k$
- *where N is the number of output nodes. However, suppose that*
- *make two errors.*
- *In the first, the target is bigger than the output, while in the second the output is bigger than the target.*
- If these two errors are the same size, then if add them up we could get 0, which means that the error value suggests that no error was made.

- To get around this we need to make all errors have the same sign.
- Best way is the sum-of-squares error function, which calculates the difference between y and t for each node, squares them, and adds them all together:

$$E(\mathbf{t}, \mathbf{y}) = \frac{1}{2} \sum_{k=1}^N (y_k - t_k)^2. \quad (4.1)$$

- if we differentiate a function, then it tells the gradient of that function, which is the direction along which it increases and decreases the most.
- So if we differentiate an error function, we get the gradient of the error.
- Since the purpose of learning is to minimise the error, following the error function downhill.
- Imagine a ball rolling around on a surface that looks like the line in Figure 4.3. Gravity will make the ball roll downhill (follow the downhill gradient) until it ends up in the bottom of one of the hollows.

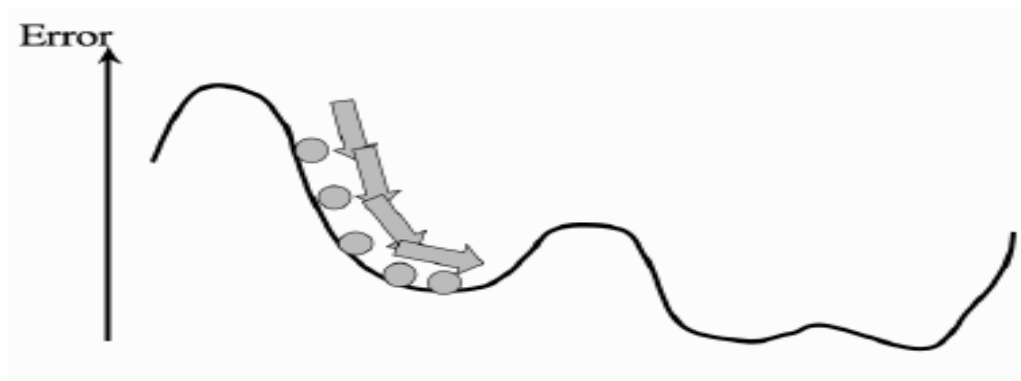


FIGURE 4.3 The weights of the network are trained so that the error goes downhill until it reaches a local minimum, just like a ball rolling under gravity.

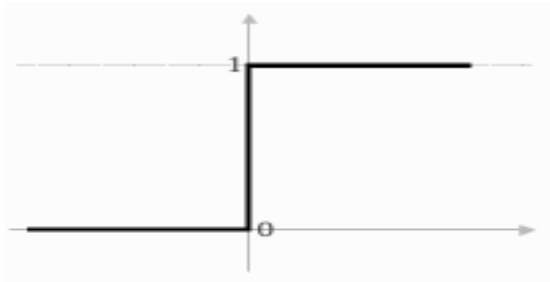


FIGURE 4.4 The threshold function that we used for the Perceptron. Note the discontinuity where the value changes from 0 to 1.

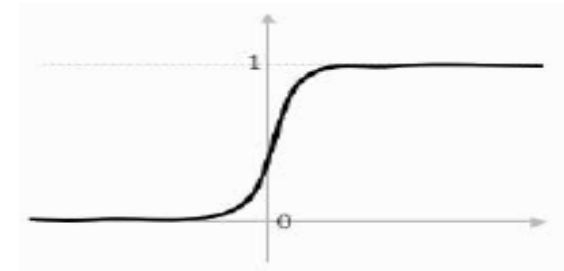


FIGURE 4.5 The sigmoid function, which looks qualitatively fairly similar, but varies smoothly and differentially.

- These are places where the error is small, so that is exactly what we want.
- This is why the algorithm is called gradient descent.

- There are only three things in the network that change: the inputs, the activation function that decides whether or not the node fires, and the weights.
- The first and second are out of our control when the algorithm is running, so only the weights matter, and therefore they are what we differentiate with respect to.
- The problem is that we need that jump between firing and not firing to make it act like a neuron.
- We can solve the problem if we can find an activation function that looks like a threshold function, but is differentiable so that we can compute the gradient.
- If you squint at a graph of the threshold function (for example, Figure 4.4) then it looks kind of S-shaped.
- There is a mathematical form of S-shaped functions, called sigmoid functions (see Figure 4.5).
- The most commonly used form of this function (where *is some positive parameter*) is:

$$a = g(h) = \frac{1}{1 + \exp(-\beta h)}. \quad (4.2)$$

- In some texts you will see the activation function given a different form, as:

$$a = g(h) = \tanh(h) = \frac{\exp(h) - \exp(-h)}{\exp(h) + \exp(-h)}, \quad (4.3)$$

- which is the hyperbolic tangent function.
- This is a different but similar function; it is still a sigmoid function, but it saturates (reaches its constant values) at ± 1 instead of 0 and 1, which is sometimes useful. It also has a relatively simple derivative:

$$\frac{d}{dx} \tanh x = (1 - \tanh^2(x)).$$

- We can convert between the two easily, because if the saturation points are (± 1) , then we can convert to $(0, 1)$ by using $0.5 \times (x + 1)$.
- Got a new form of error computation and a new activation function that decides whether or not a neuron should fire

- Compute the gradient of these errors and use them to decide how much to update each weight in the network.
- We will do that first for the nodes connected to the output layer, and after we have updated those, we will work *backwards through the network until we get back to the inputs* again.
- There are just two problems:
 - for the output neurons, we don't know the inputs.
 - for the hidden neurons, we don't know the targets; for extra hidden layers, we know neither the inputs nor the targets, but even this won't matter for the algorithm we derive.
- The chain rule of differentiation tells us that if we want to know how the error changes as we vary the weights, we can think about how the error changes as we vary the inputs to the weights, and multiply this by how those input values change as we vary the weights.
- This is useful because it lets us calculate all of the derivatives that we want to:

- we can write the activations of the output nodes in terms of the activations of the hidden nodes and the output weights, and then we can send the error calculations back through the network to the hidden layer to decide what the target outputs were for those neurons.
- Do exactly the same computations if the network has extra hidden layers between the inputs and the outputs.
- It gets harder to keep track of which functions we should be differentiating, but there are no new tricks needed.
- Compute the gradients of the errors with respect to the weights, so that we change the weights so that we go downhill, which makes the errors get smaller.
- We do this by differentiating the error function with respect to the weights, but we can't do this directly, so we have to apply the chain rule and differentiate with respect to things that we know.

- This leads to two different update functions, one for each of the sets of weights, and we just apply these backwards through the network, starting at the outputs and ending up back at the inputs.

The Multi-layer Perceptron Algorithm

- Look at some practical issues, such as how much training data is needed, how much training time is needed, and how to choose the correct size of network.
- We will assume that there are L input nodes, plus the bias, M hidden nodes, also plus a bias, and N output nodes, so that there are $(L+1) \times M$ weights between the input and the hidden layer and $(M+1) \times N$ between the hidden layer and the output.
- The sums will start from 0 if they include the bias nodes and 1 otherwise, and run up to L, M , or N , so that $x_0 = -1$ is the bias input, and $a_0 = -1$ is the bias hidden node.

- The algorithm that is described could have any number of hidden layers, in which case there might be several values for M , and extra sets of weights between the hidden layers.
- We will also use i, j, k to index the nodes in each layer in the sums, and the corresponding Greek letters (L, ζ, K) for fixed indices.
- Quick summary of how the algorithm works, and then the full MLP training algorithm using back-propagation of error is described.
- 1. an input vector is put into the input nodes
- 2. the inputs are fed forward through the network (Figure 4.6)
 - the inputs and the first-layer weights (here labelled as v) are used to decide whether the hidden nodes fire or not. The activation function $g(\cdot)$ is the sigmoid function given in Equation (4.2) above
 - the outputs of these neurons and the second-layer weights (labelled as w) are used to decide if the output neurons fire or not

- 3. The *error is computed as the sum-of-squares difference between the network outputs and the targets*
- 4. this error is fed *backwards through the network in order to*
 - first update the second-layer weights
 - and then afterwards, the first-layer weights

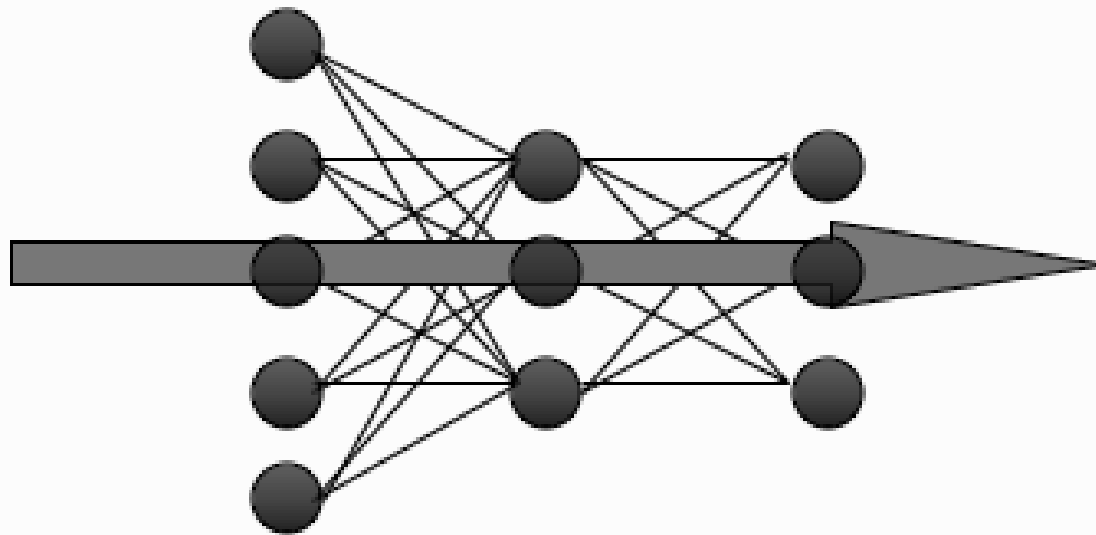


FIGURE 4.6 The forward direction in a Multi-layer Perceptron.

The Multi-layer Perceptron Algorithm

- Initialisation

- initialise all weights to small (positive and negative) random values

- Training

- repeat:

- * for each input vector:

- Forwards phase:

- compute the activation of each neuron j in the hidden layer(s) using:

$$h_{\zeta} = \sum_{i=0}^L x_i v_{i\zeta} \quad (4.4)$$

$$a_{\zeta} = g(h_{\zeta}) = \frac{1}{1 + \exp(-\beta h_{\zeta})} \quad (4.5)$$

- work through the network until you get to the output layer neurons, which have activations (although see also Section 4.2.3):

$$h_{\kappa} = \sum_j a_j w_{j\kappa} \quad (4.6)$$

$$y_{\kappa} = g(h_{\kappa}) = \frac{1}{1 + \exp(-\beta h_{\kappa})} \quad (4.7)$$

Backwards phase:

- compute the error at the output using:

$$\delta_o(\kappa) = (y_\kappa - t_\kappa) y_\kappa (1 - y_\kappa) \quad (4.8)$$

- compute the error in the hidden layer(s) using:

$$\delta_h(\zeta) = a_\zeta (1 - a_\zeta) \sum_{k=1}^N w_{\zeta} \delta_o(k) \quad (4.9)$$

- update the output layer weights using:

$$w_{\zeta\kappa} \leftarrow w_{\zeta\kappa} - \eta \delta_o(\kappa) a_\zeta^{\text{hidden}} \quad (4.10)$$

- update the hidden layer weights using:

$$v_\iota \leftarrow v_\iota - \eta \delta_h(\kappa) x_\iota \quad (4.11)$$

- * (if using sequential updating) randomise the order of the input vectors so that you don't train in exactly the same order each iteration

- until learning stops (see Section 4.3.3)

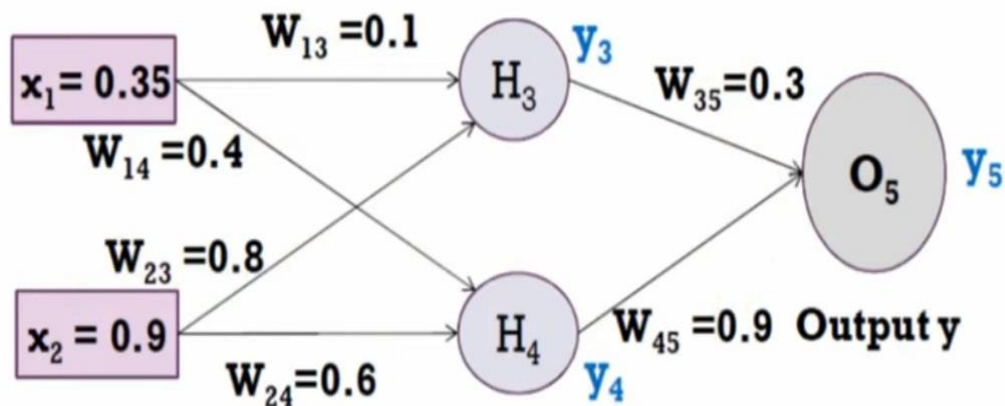
- **Recall**

- use the Forwards phase in the training section above

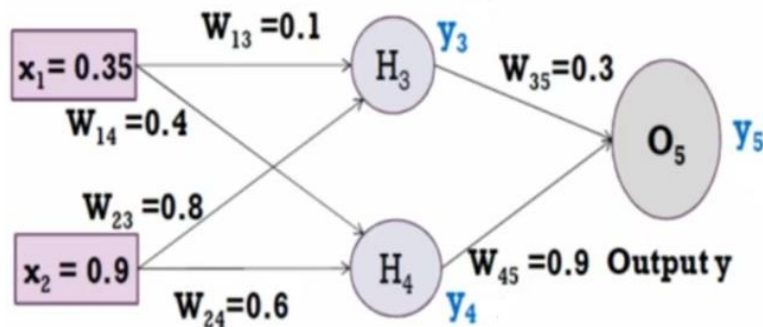
- Sometimes, even 5000 iterations are not enough for the XOR function, and more have to be added.

Multi-layer Perceptron Example

Assume that the neurons have a sigmoid activation function, perform a forward pass and a backward pass on the network. Assume that the actual output of y is 0.5 and learning rate is 1. Perform another forward pass.



Multi-layer Perceptron Example



- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_i (w_{ij} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$a_1 = (w_{13} * x_1) + (w_{23} * x_2) \\ = (0.1 * 0.35) + (0.8 * 0.9) = 0.755$$

$$y_3 = f(a_1) = 1 / (1 + e^{-0.755}) = 0.68$$

$$a_2 = (w_{14} * x_1) + (w_{24} * x_2) \\ = (0.4 * 0.35) + (0.6 * 0.9) = 0.68$$

$$y_4 = f(a_2) = 1 / (1 + e^{-0.68}) = 0.6637$$

$$a_3 = (w_{35} * y_3) + (w_{45} * y_4) \\ = (0.3 * 0.68) + (0.9 * 0.6637) = 0.801$$

$$y_5 = f(a_3) = 1 / (1 + e^{-0.801}) = \mathbf{0.69 \text{ (Network Output)}}$$

$$\text{Error} = y_{\text{target}} - y_5 = -0.19$$

Multi-layer Perceptron Example

- Each weight changed by:

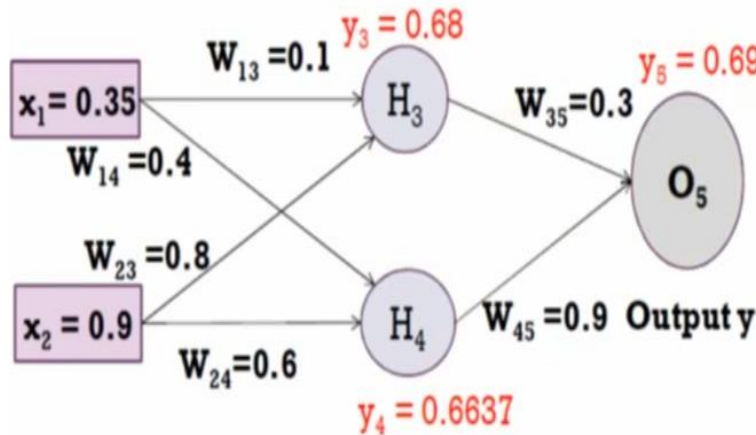
$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\delta_j = o_j(1 - o_j)(t_j - o_j) \quad \text{if } j \text{ is an output unit}$$

$$\delta_j = o_j(1 - o_j) \sum_k \delta_k w_{kj} \quad \text{if } j \text{ is a hidden unit}$$

- where η is a constant called the learning rate
- t_j is the correct teacher output for unit j
- δ_j is the error measure for unit j

Multi-layer Perceptron Example



- Backward Pass: Compute δ_3 , δ_4 and δ_5 .

For output unit:

$$\delta_5 = y(1-y) (y_{\text{target}} - y) \\ = 0.69*(1-0.69)*(0.5-0.69) = -0.0406$$

For hidden unit:

$$\delta_3 = y_3(1-y_3) w_{35} * \delta_5 \\ = 0.68*(1 - 0.68)*(0.3 * -0.0406) = -0.00265$$

Compute new weights

$$\Delta w_{ji} = \eta \delta_j o_i$$

$$\Delta w_{45} = \eta \delta_5 y_4 = 1 * -0.0406 * 0.6637 = -0.0269 \\ w_{45} \text{ (new)} = \Delta w_{45} + w_{45} \text{ (old)} = -0.0269 + (0.9) = 0.8731$$

$$\delta_4 = y_4(1-y_4) w_{45} * \delta_5 \\ = 0.6637*(1 - 0.6637)* (0.9 * -0.0406) = -0.0082$$

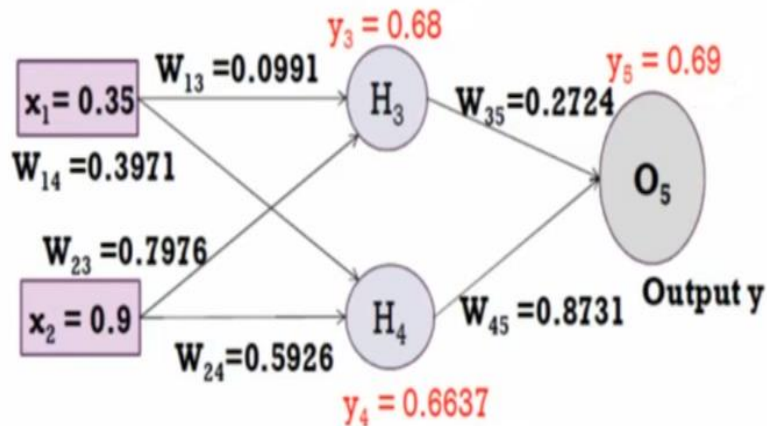
$$\Delta w_{14} = \eta \delta_4 x_1 = 1 * -0.0082 * 0.35 = -0.00287 \\ w_{14} \text{ (new)} = \Delta w_{14} + w_{14} \text{ (old)} = -0.00287 + 0.4 = 0.3971$$

Multi-layer Perceptron Example

- Similarly, update all other weights

i	j	w_{ij}	δ_j	x_i	η	Updated w_{ij}
1	3	0.1	-0.00265	0.35	1	0.0991
2	3	0.8	-0.00265	0.9	1	0.7976
1	4	0.4	-0.0082	0.35	1	0.3971
2	4	0.6	-0.0082	0.9	1	0.5926
3	5	0.3	-0.0406	0.68	1	0.2724
4	5	0.9	-0.0406	0.6637	1	0.8731

Multi-layer Perceptron Example



- Forward Pass: Compute output for y_3 , y_4 and y_5 .

$$a_j = \sum_j (w_{i,j} * x_i) \quad y_j = F(a_j) = \frac{1}{1 + e^{-a_j}}$$

$$\begin{aligned} a_1 &= (w_{13} * x_1) + (w_{23} * x_2) \\ &= (0.0991 * 0.35) + (0.7976 * 0.9) = 0.7525 \\ y_3 &= f(a_1) = 1 / (1 + e^{-0.7525}) = 0.6797 \end{aligned}$$

$$\begin{aligned} a_2 &= (w_{14} * x_1) + (w_{24} * x_2) \\ &= (0.3971 * 0.35) + (0.5926 * 0.9) = 0.6723 \\ y_4 &= f(a_2) = 1 / (1 + e^{-0.6723}) = 0.6620 \end{aligned}$$

$$\begin{aligned} a_3 &= (w_{35} * y_3) + (w_{45} * y_4) \\ &= (0.2724 * 0.6797) + (0.8731 * 0.6620) = 0.7631 \\ y_5 &= f(a_3) = 1 / (1 + e^{-0.7631}) = \mathbf{0.6820 \text{ (Network Output)}} \end{aligned}$$

$$\text{Error} = y_{\text{target}} - y_5 = -0.182$$

Initialising the Weights

- The MLP algorithm suggests that the weights are initialised to small random numbers, both positive and negative.
- One way to get a feeling for this would be to experiment with the code, setting all of the weights to 0, and seeing how well the network learns, then setting them all to large numbers and comparing the results.
- If the initial weight values are close to 1 or -1 (which is what we mean by large here) then the inputs to the sigmoid are also likely to be close to ± 1 and so the output of the neuron is either 0 or 1 (the sigmoid has saturated, reached its maximum or minimum value).
- If the weights are very small (close to zero) then the input is still close to 0 and so the output of the neuron is just linear, so we get a linear model

- So a common trick is to set the weights in the Range $-1/\sqrt{n} < w < 1/\sqrt{n}$, where n is the number of nodes in the input layer to those weights. This makes the total input to a neuron have a maximum size of about 1.
- if the weights are large, then the activation of a neuron is likely to be at, or close to, 0 or 1 already, which means that the gradients are small, and so the learning is very slow.
- There is an interplay here with the value of β in the logistic function, which means that small values of β (say $\beta = 3.0$ or less) are more effective.
- Random values are used for the initialisation so that the learning starts off from different places for each run, and we keep them all about the same size because we want all of the weights to reach their final values at about the same time.
- This is known as **uniform learning** and it is important because otherwise the network will do better on some inputs than others.

Different Output Activation Functions

- Sigmoid neurons are used in the hidden layer and the output layer.
- This is fine for classification problems, since there we can make the classes be 0 and 1.
- To perform regression problems, where the output needs to be from a continuous range, not just 0 or 1.
- The sigmoid neurons at the output are not very useful in that case.
- Replace the output neurons with linear nodes that just sum the inputs and give that as their activation.
- This does not mean that we change the hidden layer neurons; they stay exactly the same, and we only modify the output nodes.
- They are not models of neurons anymore, since they don't have the characteristic fire/don't fire pattern.

- Even so, they enable us to solve regression problems, where we want a real number out, not just a 0/1 decision.
- A third type of output neuron that is also used, which is the soft-max activation function.
- This is most commonly used for classification problems where the 1-of- N *output* encoding is used.
- The soft-max function rescales the outputs by calculating the exponential of the inputs to that neuron, and dividing by the total sum of the inputs to all of the neurons, so that the activations sum to 1 and all lie between 0 and 1.
- As an activation function it can be written as:

$$y_k = g(h_k) = \frac{\exp(h_k)}{\sum_{k=1}^N \exp(h_k)}. \quad (4.12)$$

THE MULTI-LAYER PERCEPTRON IN PRACTICE

- MLP is used to find solutions to four different types of problem:
 - regression,
 - classification,
 - time-series prediction, and
 - data compression

AMOUNT OF TRAINING DATA

- For the MLP with one hidden layer there are $(L + 1) \times M + (M + 1) \times N$ weights, where L, M, N are the number of nodes in the input, hidden, and output layers, respectively.
- The extra +1s come from the bias nodes, which also have adjustable weights.
- Setting the values of these weights is the job of the back-propagation algorithm, which is driven by the errors coming from the training data.
- The more training data there is, the better for learning, although the time that the algorithm takes to learn increases.

- Number of training examples used is at least 10 times the number of weights.
- Neural network training is a fairly computationally expensive operation.

NUMBER OF HIDDEN LAYERS

- There are two other considerations concerning the number of weights that are inherent in the calculation above, which is the choice of the number of hidden nodes, and the number of hidden layers.
- Normally see a pictorial demonstration of the fact that two hidden layers is the most that you ever need for normal MLP learning.
- In fact it is possible to show mathematically that one hidden layer with lots of hidden nodes is sufficient.
- This is known as the **Universal Approximation Theorem**
- There is no theory to guide the choice of the number of hidden nodes.

- Have to experiment by training networks with different numbers of hidden nodes and then choosing the one that gives the best results
- we will never normally need more than two layers (that is, one hidden layer and the output layer). Fig.4.9
- This is because we can approximate any smooth functional mapping using a linear combination of localised sigmoidal functions.
- The basic idea is that by combining sigmoid functions we can generate ridge-like functions, and by combining ridge-like functions we can generate functions with a unique maximum.
- By combining these and transforming them using another layer of neurons, we obtain a localised response (a 'bump' function), and any functional mapping can be approximated to arbitrary accuracy using a linear combination of such bumps. Fig.4.10

- Two hidden layers are sufficient to compute these bump functions for different inputs, and so if the function that we want to learn (approximate) is continuous, the network can compute it.
- It can therefore approximate any decision boundary, not just the linear one that the Perceptron computed.

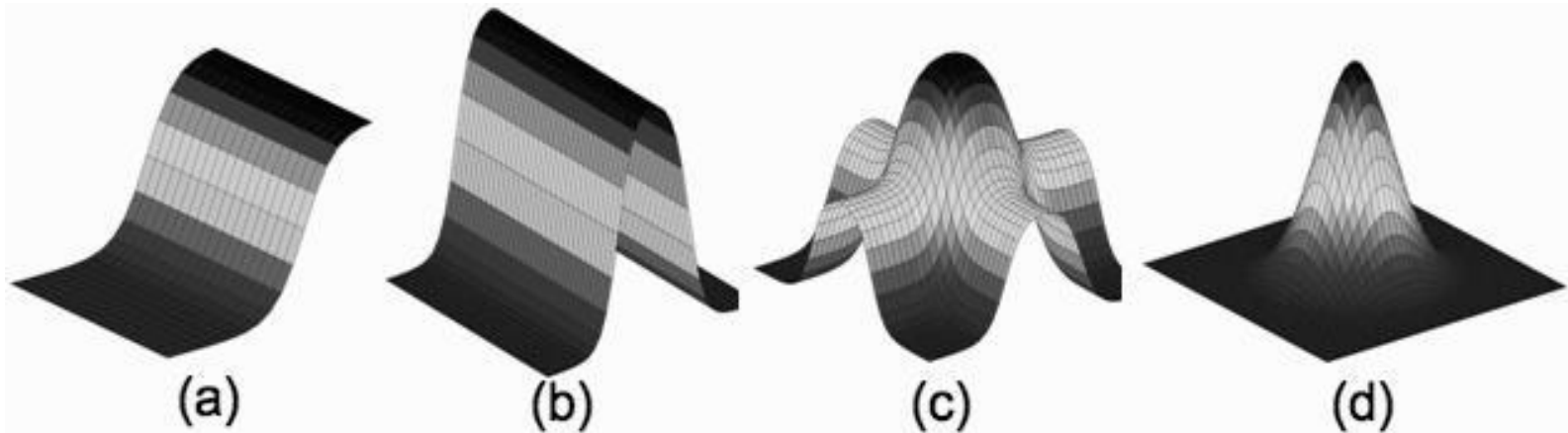


FIGURE 4.9 The learning of the MLP can be shown as the output of a single sigmoidal neuron (a), which can be added to others, including reversed ones, to get a hill shape (b). Adding another hill at 90 produces a bump (c) , which can be sharpened to any extent we want (d), with the bumps added together in the output layer. Thus the MLP learns a local representation of individual inputs.

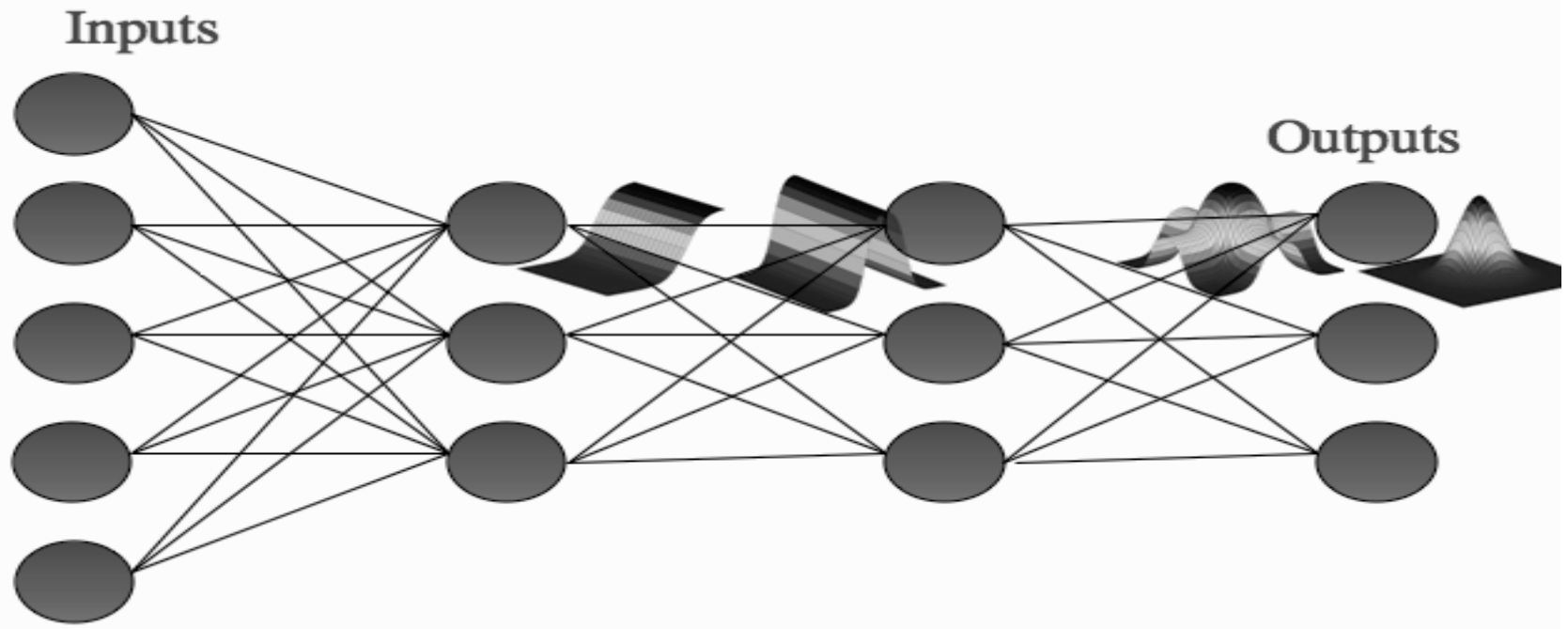


FIGURE 4.10 Schematic of the effective learning shape at each stage of the MLP.

WHEN TO STOP LEARNING

- The training of the MLP requires that the algorithm runs over the entire dataset many times, with the weights changing as the network makes errors in each iteration.
- The question is how to decide when to stop learning,

- If we plot the sum-of-squares error during training, it typically reduces fairly quickly during the first few training iterations, and then the reduction slows down as the learning algorithm performs small changes to find the exact local minimum.
- This is where the validation set comes in useful.
- We train the network for some predetermined amount of time, and then use the validation set to estimate how well the network is generalising.
- We then carry on training for a few more iterations, and repeat the whole process.
- At some stage the error on the validation set will start increasing again, because the network has stopped learning about the function that generated the data, and started to learn about the noise that is in the data itself (shown in Figure 4.11).
- At this stage we stop the training.
- This technique is called early stopping.

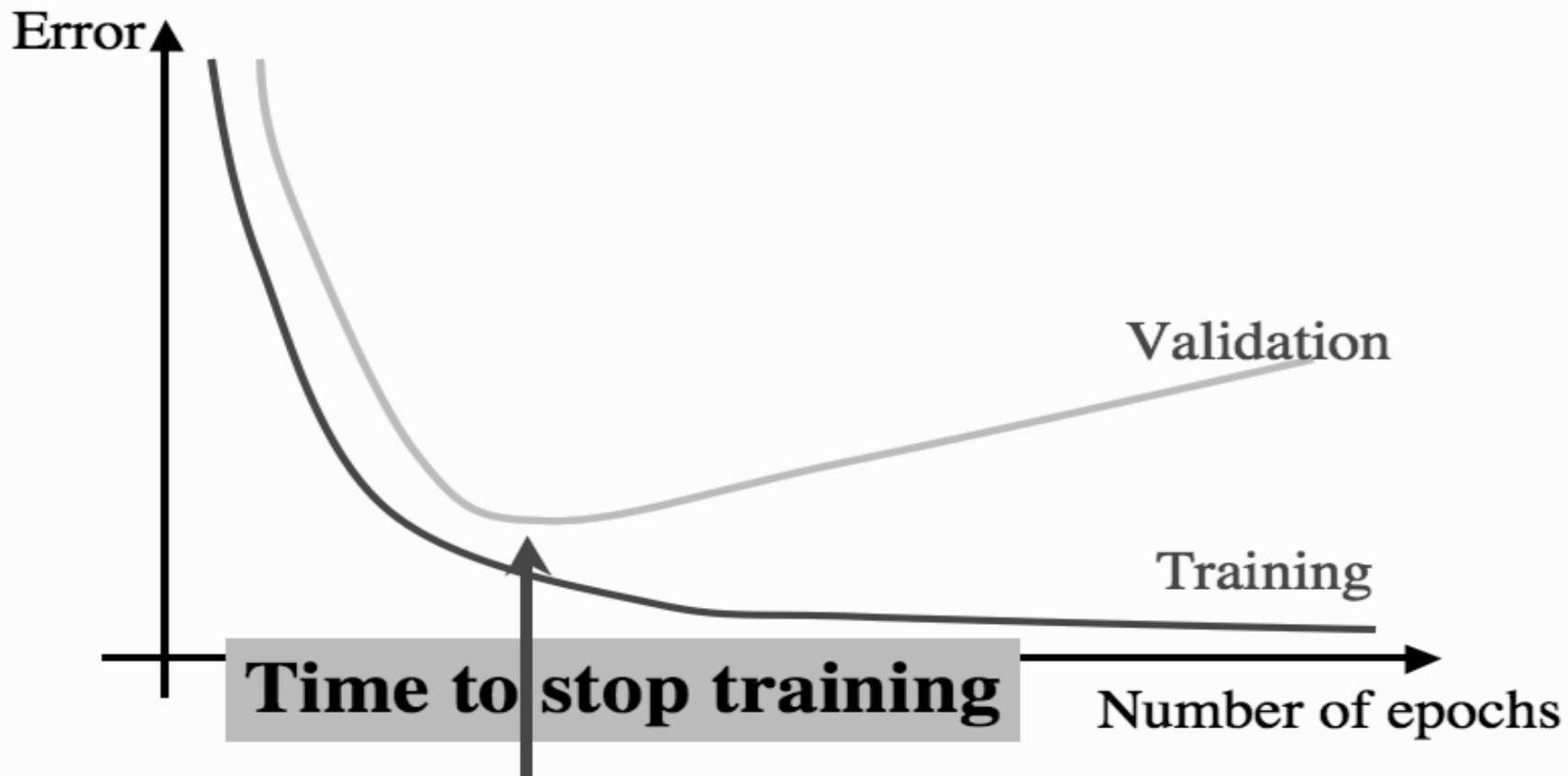


FIGURE 4.11 The effect of overfitting on the training and validation error curves, with the point at which early stopping will stop the learning marked.

EXAMPLES OF USING THE MLP

- Demonstrations of how to make the network learn on data.
- Four types of problems that are generally solved using an MLP: regression, classification, time-series prediction, and data compression/data denoising.

A REGRESSION PROBLEM

- The regression problem considered is a very simple one.
- Take a set of samples generated by a simple mathematical function, and try to learn the generating function (that describes how the data was made) so that we can find the values of any inputs, not just the ones we have training data for.
- The function used is a sine wave.
- Train an MLP on the data. There is one input value, x and one output value t , so the neural network will have one input and one output.

- Use linear neurons at the output and it should be the value of the function, rather than 0 or 1.
- Normalise the data using the method and then separate the data into training, testing, and validation sets.
- For this example there are only 40 data points, and use half of them as the training set, although it may not be enough for the algorithm to learn effectively.
- Split the data in the ratio 50:25:25 by using the odd-numbered elements as training data, the even-numbered ones that do not divide by 4 for testing, and the rest for validation.
- Construct a network with three nodes in the hidden layer, and run it for 101 iterations with a learning rate of 0.25,
- The output error decreasing means network is learning Ex:

Iteration: 0 Error: 12.3704163654 Iteration: 100 Error:
8.2075961385

- We train the network for a few iterations (let's make it 10 for now), then evaluate the validation set error by running the network forward (i.e., the recall phase).
- Learning should stop when the validation set error starts to increase.

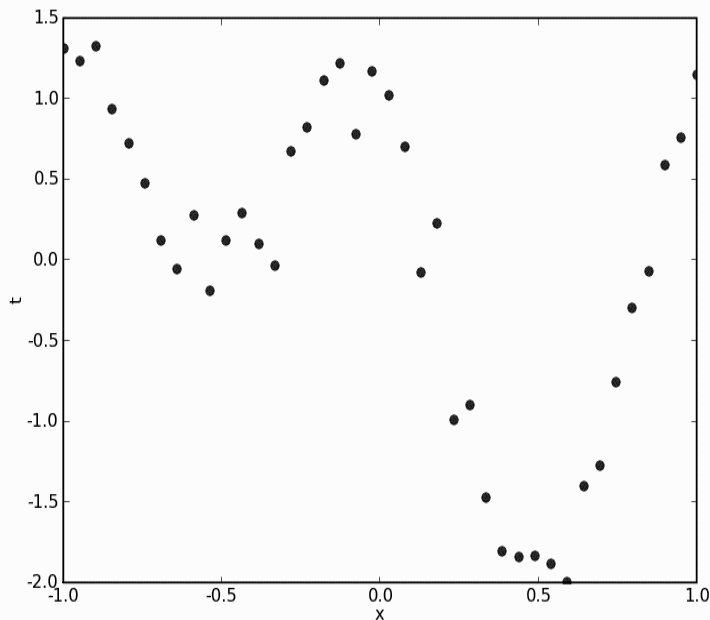


FIGURE 4.12 The data that we will learn using an MLP, consisting of some samples from a sine wave with Gaussian noise added.

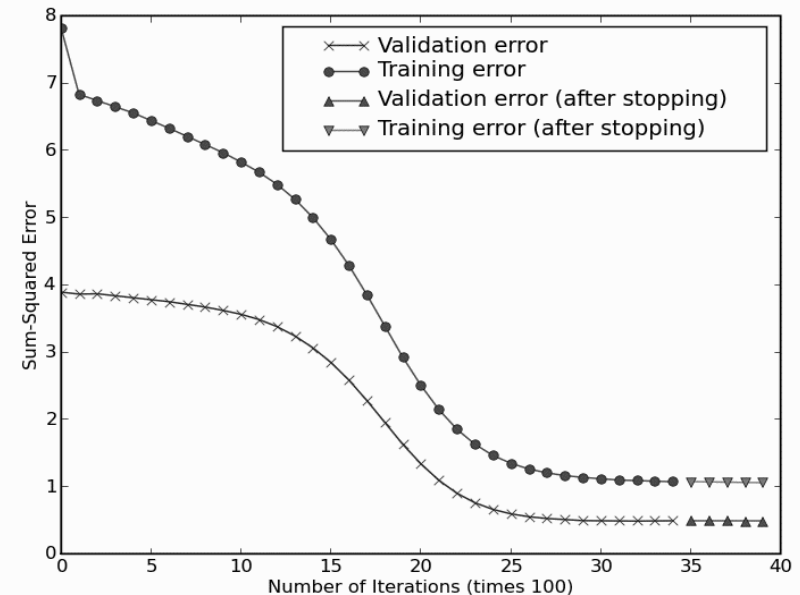


FIGURE 4.13 Plot of the error as the MLP learns (top line is total error on the training set; bottom line is on the validation set; it is larger on the training set because there are more datapoints in this set). Early-stopping halts the learning at the point where there is no line, where the crosses become triangles. The learning was continued to show that the error got slightly worse afterwards.

- So each network size is run 10 times, and the average is monitored.
- The following table shows the results of doing this, reporting the sum-of-squares validation error, for a few different sizes of network

No. of hidden nodes	1	2	3	5	10	25	50
Mean error	2.21	0.52	0.52	0.52	0.55	1.35	2.56
Standard deviation	0.17	0.00	0.00	0.02	0.00	1.20	1.27
Max error	2.31	0.53	0.54	0.54	0.60	3.230	3.66
Min error	2.10	0.51	0.50	0.50	0.47	0.42	0.52

- Based on these numbers, we would select a network with a small number of hidden nodes, certainly between 2 and 10 (and the smaller the better, in general), since their maximum error is much smaller than a network with just 1 hidden node.

A Classification Example

- The Iris Dataset: As an example we are going to look at another example from the UCI Machine Learning repository.
- This one is concerned with classifying examples of three types of iris (flower) by the length and width of the sepals and petals and is called iris.
- It was originally worked on by R.A. Fisher, a famous statistician and biologist, who analysed it in the 1930s.
- Unfortunately we can't currently load this into NumPy using `loadtxt()` because the class (which is the last column) is text rather than a number, and the `txt` in the function name doesn't mean that it reads text, only numbers in plaintext format.
- Last column in dataset is the class ID, and the others are the four measurements.
- start by normalising the inputs, using the maximum rather than the variance, and leaving the class IDs alone.

- Need to convert the targets into 1-of- N encoding, from their current encoding as class 1, 2, or 3.
- This is pretty easy if we make a new matrix that is initially all zeroes, and simply set one of the entries to be 1.
- Separate the data into training, testing, and validation sets.
- There are 150 examples in the dataset, and they are split evenly amongst the three classes, so the three classes are the same size and we don't need to worry about discarding any data points.
- Split them into half training, and one quarter each testing and validation.
- If you look at the file, you will notice that the first 50 are class 1, the second 50 class 2, etc.
- Therefore need to randomise the order before we split them into sets, to ensure that there are not too many of one class in one of the sets finally ready to set up and train the network.

Time-Series Prediction

- There is a common data analysis task known as time-series prediction, where we have a set of data that show how something varies over time, and we want to predict how the data will vary in the future.
- It is quite a difficult task, but a fairly important one.
- To predict stock markets and disease patterns.
- The problem is that even if there is some regularity in the time-series, it can appear over many different scales.
- Ex: there is often seasonal variation—if we plotted average temperature over several years, would notice that it got hotter in the summer and colder in the winter, but might not notice if there was an overall upward or downward trend to the summer temperatures, because the summer peaks are spread too far apart in the data.

- The other problems with the data are practical. How many data points should we look at to make the prediction and how far apart in time should we space those inputs.
- write this as an equation, where predicting y using a neural network that is written as a function $f(\cdot)$:

$$y = x(t + \tau) = f(x(t), x(t - \tau), \dots, x(t - k\tau)), \quad (4.19)$$

- where the two questions about how many data points and how far apart they should be come down to choices about T and k .
- The target data for training the neural network is simple, because it comes from further up the time-series, and so training is easy.
- Suppose that $T = 2$ and $k = 3$. Then the first input data are elements 1, 3, 5 of the dataset, and the target is element 7.
- The next input vector is elements 2, 4, 6, with target 8, and then 3, 5, 7 with target 9. You train the network by passing through the time-series (remembering to save some data for testing), and then press on into the future making predictions.

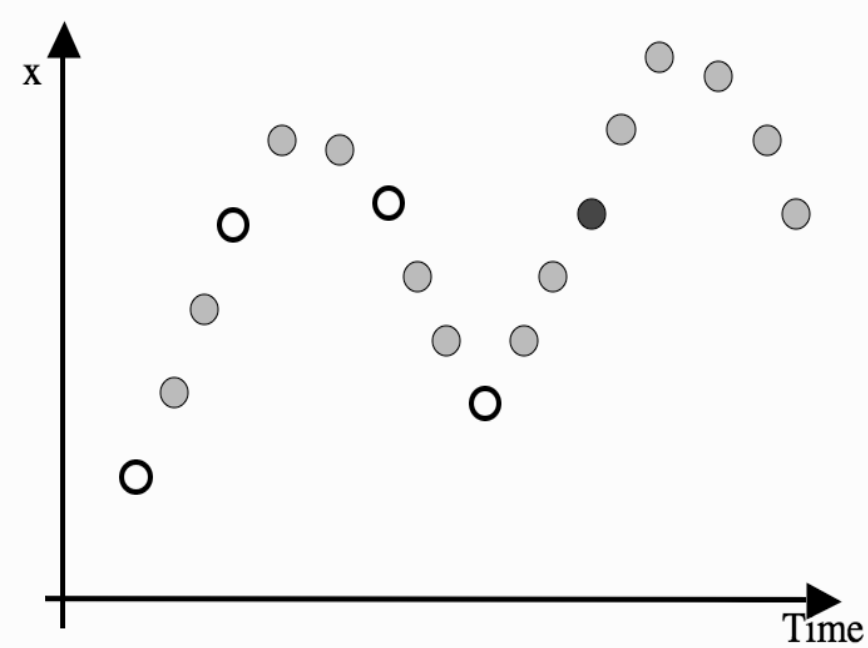


FIGURE 4.14 Part of a time-series plot, showing the datapoints and the meanings of T and k .

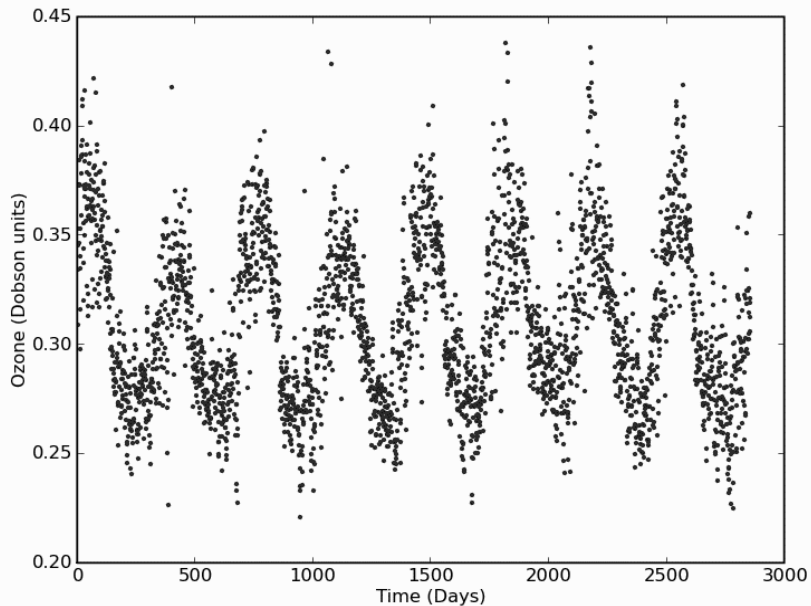


FIGURE 4.15 Plot of the ozone layer thickness above Palmerston North in New Zealand between 1996 and 2004.

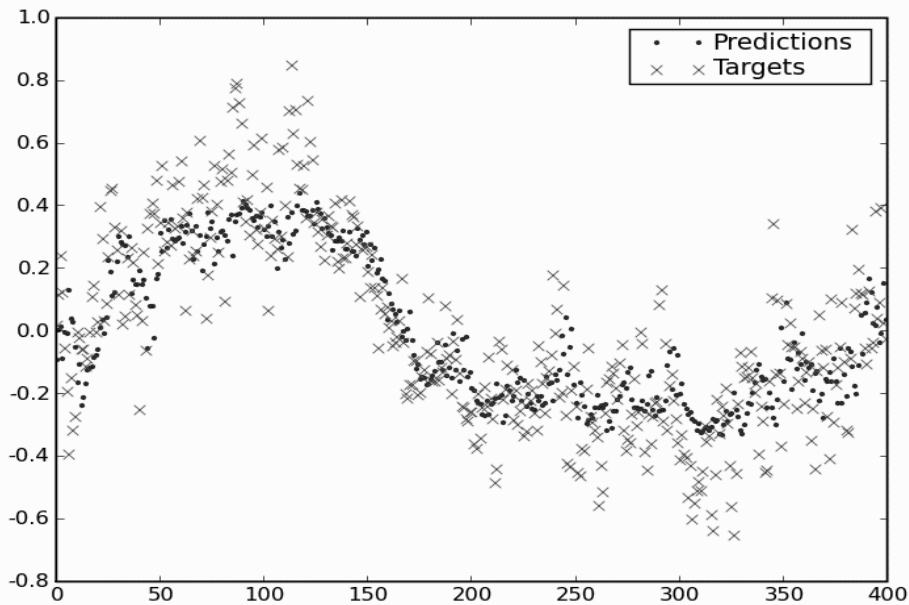


FIGURE 4.16 Plot of 400 predicted and actual output values of the ozone data using the MLP as a time-series predictor with $k = 3$ and $T = 2$.

Data Compression: The Auto-Associative Network

- Train the network to reproduce the inputs at the output layer (called auto-associative learning; sometimes the network is known as an auto encoder).
- The network is trained so that whatever you show it at the input is reproduced at the output, which doesn't seem very useful at first, but suppose that we use a hidden layer that has fewer neurons than the input layer Figure 4.17

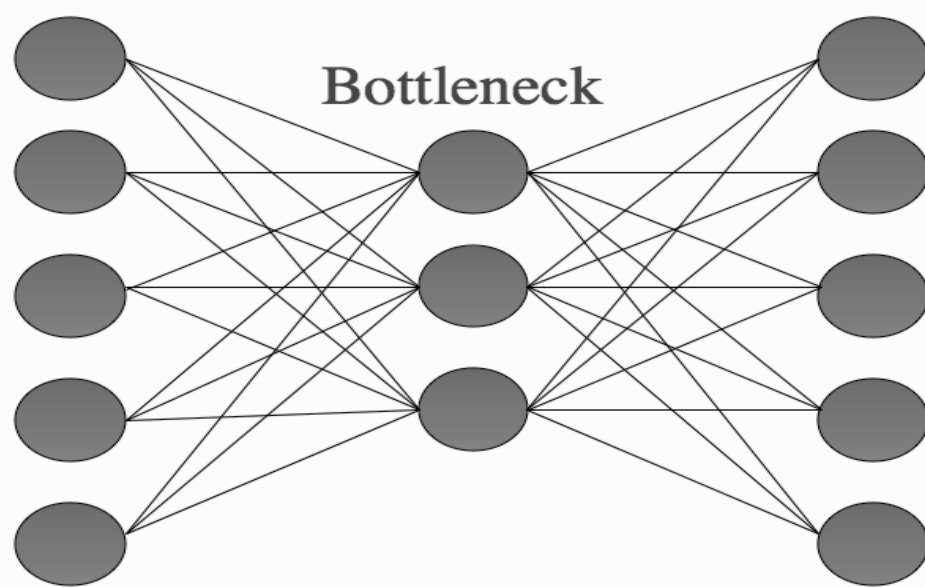


FIGURE 4.17 The auto-associative network. The network is trained to reproduce the inputs at the outputs, passing them through the bottleneck hidden layer that compresses the data.

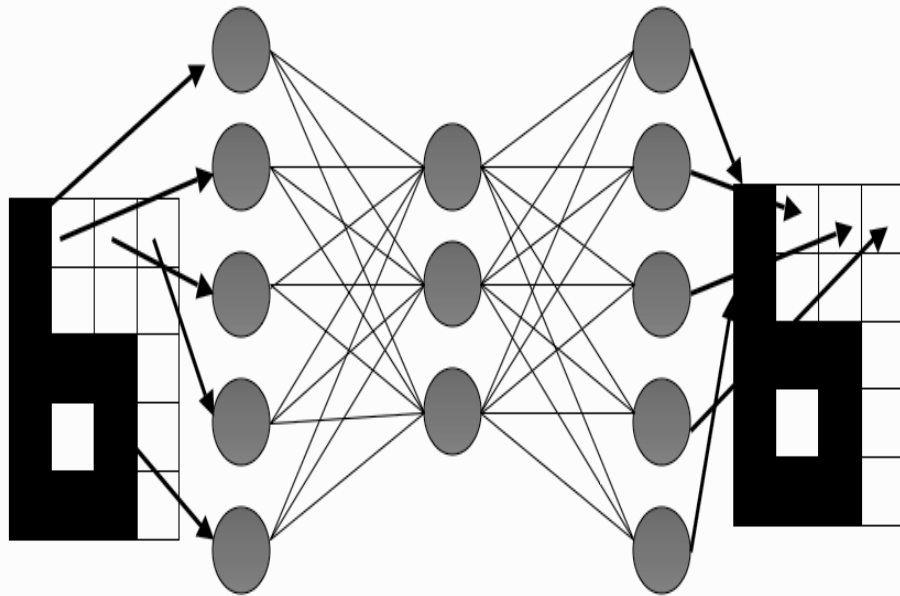


FIGURE 4.18 Schematic showing how images are fed into the auto-associative network for compression.

- This bottleneck hidden layer has to represent all of the information in the input, so that it can be reproduced at the output.
- It therefore performs some compression of the data, representing it using fewer dimensions than were used in the input.
- what the hidden layers of the MLP are doing: they are finding a different (often lower dimensional) representation of the input data that extracts important components of the data, and ignores the noise.
- This auto-associative network can be used to compress images and other data.
- A schematic of this is shown in Figure 4.18: the 2D image is turned into a 1D vector of inputs by cutting the image into strips and sticking the strips into a long line.
- The values of this vector are the intensity (colour) values of the image, and these are the input values.

- The network learns to reproduce the same image at the output, and the activations of the hidden nodes are recorded for each image.
- After training, we can throw away the input nodes and first set of weights of the network.
- If we insert some values in the hidden nodes (their activations for a particular image; see Figure 4.19), then by feeding these activations forward through the second set of weights, the correct image will be reproduced on the output.
- So all we need to store are the set of second-layer weights and the activations of the hidden nodes for each image, which is the compressed version.
- Auto-associative networks can also be used to denoise images, since, after training, the network will reproduce the trained image that best matches the current (noisy) input.

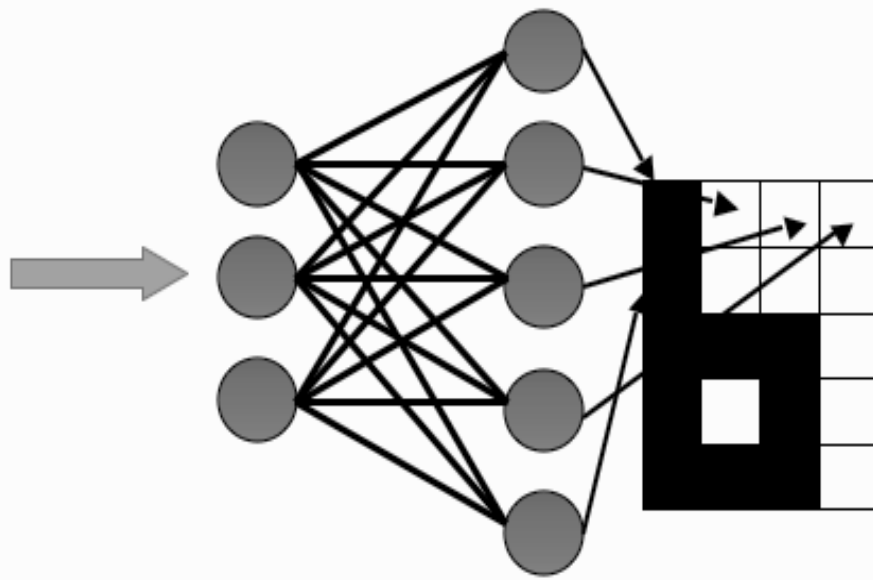


FIGURE 4.19 Schematic showing how the hidden nodes and second layer of weights can be used to regain the compressed images after the network has been trained.

- In fact, if the nodes all have linear activation, then what the network learns to compute are the Principal Components of the input data.
- Principal Components Analysis (PCA) is a useful dimensionality reduction technique

DERIVING BACK-PROPAGATION

- Derivation of the back-propagation algorithm.
- Important to understand how and why the algorithm works.
- there are only three things that you really need to know.
 - One is the derivative (with respect to x) of $\frac{1}{2} X^2$, which is x , and
 - another is the chain rule, which says that $\frac{dy}{dx} = \frac{dy}{dt} \frac{dt}{dx}$
 - The third thing is very simple $\frac{dy}{dx} = 0$ if y is not a function of x
- *With those three things clear in your mind, just follow through the algebra, and you'll be fine.*

The Network Output and the Error

- The output of the neural network is a function of three things:
 - the current input ($\mathbf{x}$)
 - the activation function $g(\cdot)$ of the nodes of the network
 - the weights of the network (v for the first layer and w for the 2nd

- We can't change the inputs, since they are what we are learning about, nor can we change the activation function as the algorithm learns.
- So the weights are the only things that we can vary to improve the performance of the network, i.e., to make it learn.
- However, we do need to think about the activation function, since the threshold function that used for the Perceptron is not differentiable (it has a discontinuity at 0).
- Run the algorithm forwards, so that we have fed the inputs (x) into the algorithm, used the first set of weights (v) to compute the activations of the hidden neurons, then those activations and the second set of weights (w) to compute the activations of the output neurons, which are the outputs of the network (y).
- Use i to be an index over the input nodes, j to be an index over the hidden layer neurons, and k to be an index over the output neurons.

The Error of the Network

Discussed the Perceptron learning rule by minimising the error function

$$E = \sum_{k=1}^N y_k - t_k.$$

- Invented a learning rule that made this error smaller.
- Everything is computed from the principles of gradient descent.
- The difference between the outputs y and the targets t , to write it as $E(v, w)$ to remind us that the only things that we can change are the weights v and w , and that changing the weights changes the output, which in turn changes the error.
- For the Perceptron we computed the error as $E = \sum_{k=1}^N y_k - t_k.$
- But there are some problems with this: if $t_k > y_k$, then the sign of the error is different to when $y_k > t_k$, so if we have lots of output nodes that are all wrong, but some have positive sign and some have negative sign, then they might cancel out.

- Choose the sum-of-squares error function, calculates the difference between y_k and t_k for each node k , squares them, and adds them together.

$$E(\mathbf{w}) = \frac{1}{2} \sum_{k=1}^N (y_k - t_k)^2 \quad (4.20)$$

$$= \frac{1}{2} \sum_{k=1}^N \left[g \left(\sum_{j=0}^M w_{jk} a_j \right) - t_k \right]^2 \quad (4.21)$$

- The second line adds in the input from the hidden layer neurons and the second-layer weights to decide on the activations of the output neurons.
- For now we're going to think about the Perceptron and index the input nodes by i and the output nodes by k , so Equation (4.21) will be replaced by:

$$\frac{1}{2} \sum_{k=1}^N \left[g \left(\sum_{i=0}^L w_{ik} x_i \right) - t_k \right]^2. \quad (4.22)$$

- we can't differentiate the threshold function, which is what the Perceptron used for $g(\cdot)$, *because it has a discontinuity (sudden jump) at the threshold value*. Miss it out completely for the moment.
- Also, for the Perceptron there are no hidden neurons, and so the activation of an output neuron is just
$$y_k = \sum_{i=0}^L w_{ik} x_i$$
- where x_i is the value of an input node, and the sum runs over the number of input nodes, including the bias node.
- Use a gradient descent algorithm that adjusts each weight w_{ik} for fixed values of x_i and y_k , in the direction of the negative gradient of $E(w)$.
- the notation $\frac{\partial}{\partial}$ means the partial derivative, and is used because there are lots of different functions that we can differentiate E with respect to: all of the different weights.

$$\begin{aligned}\frac{\partial E}{\partial w_{l\kappa}} &= \frac{\partial}{\partial w_{l\kappa}} \left(\frac{1}{2} \sum_{k=1}^N (y_k - t_k)^2 \right) \\ &= \frac{1}{2} \sum_{k=1}^N 2(y_k - t_k) \frac{\partial}{\partial w_{l\kappa}} \left(y_k - \sum_{i=0}^L w_{i\kappa} x_i \right)\end{aligned}\quad (4.23)$$

$$(4.24)$$

- Now t_k is not a function of any of the weights, since it is a value given to the algorithm, so

$$\frac{\partial t_k}{\partial w_{l\kappa}} = 0$$

- for all values of k, l, κ , and the only part of $y_k = \sum_{i=0}^L w_{i\kappa} x_i$ that is a function of $w_{l\kappa}$ is when $i = l$, that is $w_{l\kappa}$ itself, which has derivative 1

$$\frac{\partial E}{\partial w_{l\kappa}} = \sum_{k=1}^N (t_k - y_k)(-x_l). \quad (4.25)$$

- idea of the weight update rule is that we follow the gradient downhill, that is, in the direction

$$-\frac{\partial E}{\partial w_{l\kappa}}$$

- So the weight update rule (when we include the learning rate η) is:

$$w_{lK} \leftarrow w_{lK} + \eta(t_K - y_K)x_l, \quad (4.26)$$

- Note that we are computing y *differently*: for the Perceptron we used the threshold activation function, whereas in the work above we ignored the threshold function.
- This isn't very useful if we want units that act like neurons, because neurons either fire or do not fire, rather than varying continuously.

REQUIREMENTS OF AN ACTIVATION FUNCTION

In order to model a neuron we want an activation function that has the following properties:

- it must be differentiable so that we can compute the gradient
- it should saturate (become constant) at both ends of the range, so that the neuron either fires or does not fire
- it should change between the saturation values fairly quickly in the middle

- There is a family of functions called sigmoid functions because they are S-shaped (see Figure 4.5) that satisfy all those criteria perfectly.
- The form in which it is generally used is:

$$a = g(h) = \frac{1}{1 + \exp(-\beta h)}, \quad (4.27)$$

- where *is some positive parameter*.

$$g'(h) = \frac{dg}{dh} = \frac{d}{dh}(1 + e^{-\beta h})^{-1} \quad (4.28)$$

$$= -1(1 + e^{-\beta h})^{-2} \frac{de^{-\beta h}}{dh} \quad (4.29)$$

$$= -1(1 + e^{-\beta h})^{-2} (-\beta e^{-\beta h}) \quad (4.30)$$

$$= \frac{\beta e^{-\beta h}}{(1 + e^{-\beta h})^2} \quad (4.31)$$

$$= \beta g(h)(1 - g(h)) \quad (4.32)$$

$$= \beta a(1 - a) \quad (4.33)$$

Back-Propagation of Error

- It is now that we'll need the chain rule

$$\frac{\partial E}{\partial w_{\zeta\kappa}} = \frac{\partial E}{\partial h_{\kappa}} \frac{\partial h_{\kappa}}{\partial w_{\zeta\kappa}}, \quad (4.34)$$

- Where $h_{\kappa} = \sum_{j=0}^M w_{j\kappa} a_{\zeta}$ is the input to output-layer neuron ; that is, the sum of the activations of the hidden-layer neurons multiplied by the relevant (second-layer) weights.
- how the error changes as we vary the input to the output neurons, and also about how those input values change as we vary the weights.
- Let's think about the second term first (in the third line we use the fact that
- for all values of j except $j = \zeta$, when it is 1):

$$\frac{\partial w_{j\kappa}}{\partial w_{\zeta\kappa}} = 0$$

$$\frac{\partial h_{\kappa}}{\partial w_{\zeta\kappa}} = \frac{\partial \sum_{j=0}^M w_{j\kappa} a_j}{\partial w_{\zeta\kappa}} \quad (4.35)$$

$$= \sum_{j=0}^M \frac{\partial w_{j\kappa} a_j}{\partial w_{\zeta\kappa}} \quad (4.36)$$

$$= a_{\zeta}. \quad (4.37)$$

Finally reduce to the form

$$\begin{aligned} w_{\zeta\kappa} &\leftarrow w_{\zeta\kappa} - \eta \frac{\partial E}{\partial w_{\zeta\kappa}} \\ &= w_{\zeta\kappa} - \eta \delta_o(\kappa) a_{\zeta}. \end{aligned} \quad (4.46)$$

The Output Activation Functions

the three functions are:

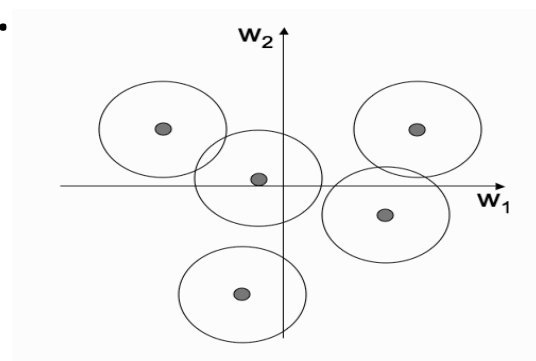
Linear $y_{\kappa} = g(h_{\kappa}) = h_{\kappa}$

Sigmoidal $y_{\kappa} = g(h_{\kappa}) = 1/(1 + \exp(-\beta h_{\kappa}))$

Soft-max $y_{\kappa} = g(h_{\kappa}) = \exp(h_{\kappa}) / \sum_{k=1}^N \exp(h_k)$

Radial Basis Functions and Splines

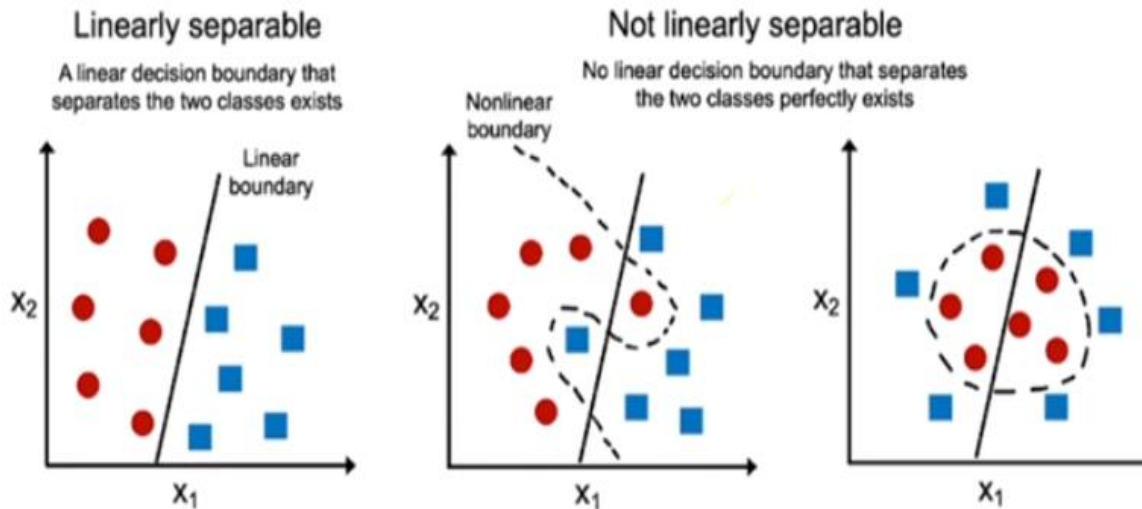
- Radial Basis Functions are real valued functions that use supervised Machine learning to perform Non linear Classification.
- supervised one, the radial basis function (RBF) network. Fig shows a set of nodes that represent radial bases in weight space.
- They are often known as centres because they each form the centres of their own circle or ellipse.



- The effect of radial basis functions in weight space. The points show the position of the RBF in weight space, while the circle around each point shows the receptive field of the node. In higher dimensions these circles become hyperspheres.

Radial Basis Functions

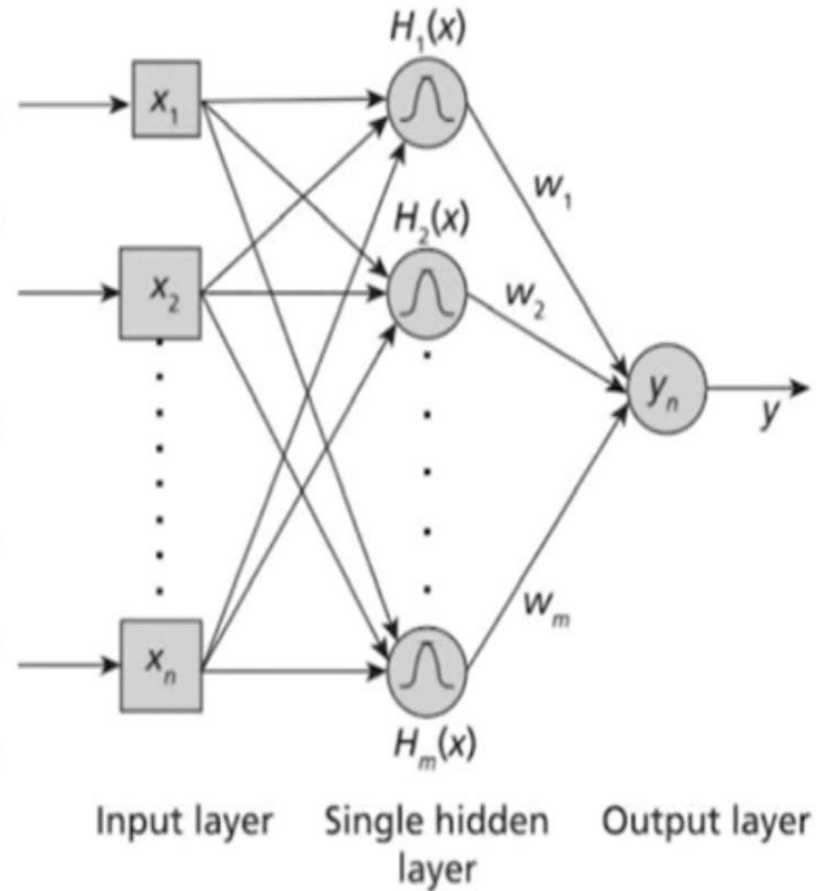
- Data can be either linearly Separable or Non-Linearly Separable
- Single layer perceptron can be used for classifying linearly Separable data.
- Multi layer perceptron is required for classifying Non-linearly Separable data.



Radial Basis Functions

Radial Basis Function is a type of Multi Layer Perceptron which has one input layer, one output layer and with strictly one hidden layer.

The hidden layer uses a non-linear radial basis function as the activation function, which converts the input parameters into high dimension space which is then fed into the network to linearly separate the problem.



Radial Basis Functions

Typical Radial Basis Functions (RBF) are:

The **Gaussian RBF** which monotonically decreases with distance from the center.

$$H(x) = e^{\frac{-(x-c)^2}{r^2}}$$

where, c is the center and r is the radius.

A **Multiquadric RBF** which monotonically increases with distance from the center.

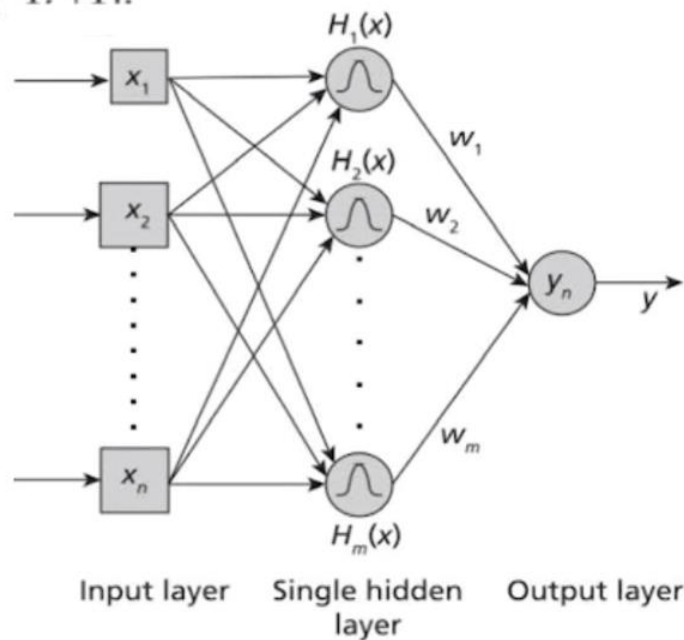
$$H(x) = \sqrt{\frac{r^2 + (x - c)^2}{r}}$$

Radial Basis Function Neural Network Algorithm

Input: Input vector $(x_1, x_2, \dots, x_n)$

Output: Y_n

Assign random weights for every connection from the Hidden layer to the Output layer in the network in the range $[-1, +1]$.



Radial Basis Function Neural Network Algorithm

Input: Input vector $(x_1, x_2, \dots, x_n)$

Output: Y_n

Assign random weights for every connection from the Hidden layer to the Output layer in the network in the range $[-1, +1]$.

Forward Phase:

Step 1: Calculate Input and Output in the Input Layer:

(Input layer is a direct transfer function, where the output of the node equals the input).

Input at Node i ' I_i ' in the Input Layer is

$$I_i = x_i$$

where,

x_i is the input received at Node i .

Output at Node i ' O_i ' in the Input Layer is

$$O_i = I_i$$

Radial Basis Function Neural Network Algorithm

Step 2: For each node j in the Hidden Layer, find the centre/receptor c and the variance r .

Define hidden layer neurons with Gaussian RBF whose output is:

$$H_j(x) = e^{\frac{-(x-c_j)^2}{r^2}}$$

where, x is the input, c_j is the centre and r is the radius.

Compute $(x - c_j)^2$ applying Euclidean distance measure between x and c_j .

Step 3: For each node k in the Output Layer, compute linear weighted sum of the output of each neuron k from the hidden layer neurons j .

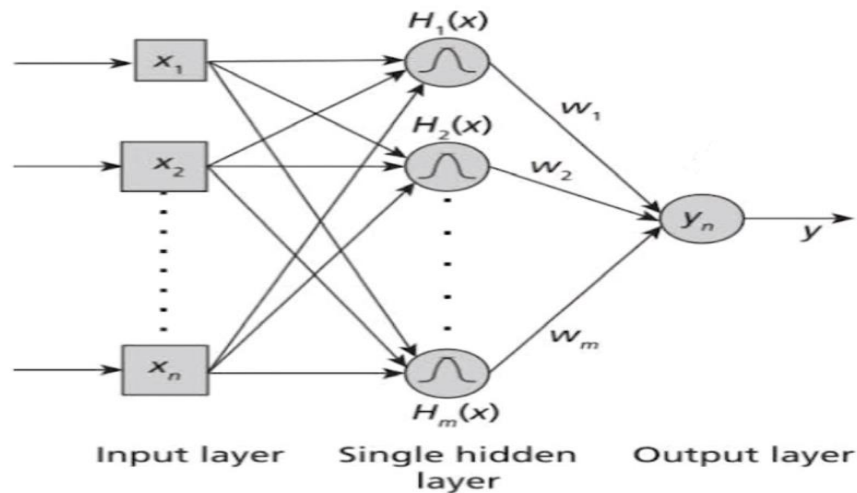
$$F_k(x) = \sum_{j=1}^m w_{jk} H_j(x)$$

where,

w_{jk} is the weight in the link from the Hidden Layer neuron j to the Output Layer neuron k .

$H_j(x)$ is the output of a Hidden Layer neuron j for an input vector x .

Radial Basis Function Neural Network Algorithm



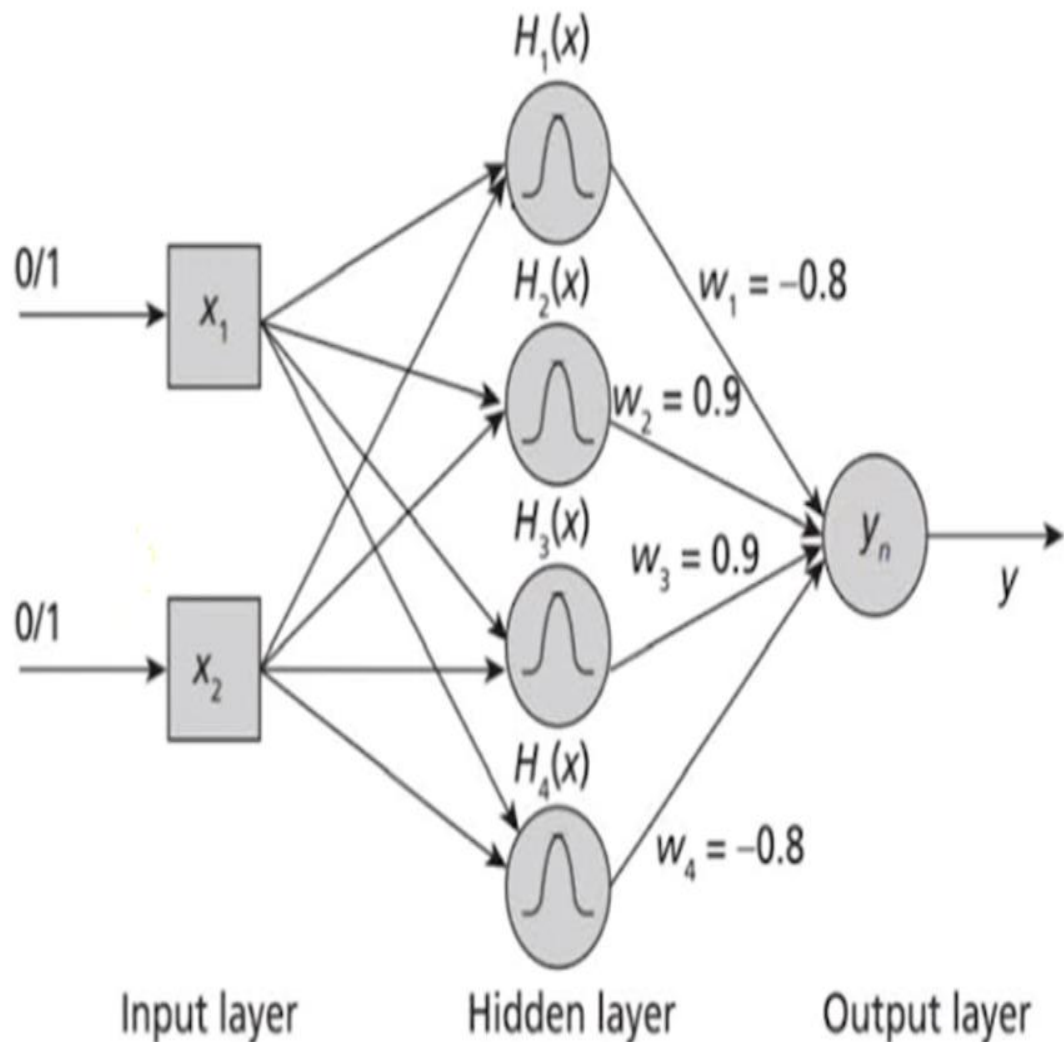
Backward Phase:

Step 1: Train the Hidden layer using Back propagation.

Step 2: Update the weights between the Hidden layer and Output layer.

Radial Basis Function Solved Example

- Consider the XOR Boolean function that has 4 patterns (0, 0) (0, 1) (1, 0) and (1, 1) in a 2-D input space.
- Radius $r = 1.414$
- Construct a RBFNN as shown in Figure that classifies the input pattern:
- (0, 0) $\rightarrow$ 0
- (0, 1) $\rightarrow$ 1
- (1, 0) $\rightarrow$ 1
- (1, 1) $\rightarrow$ 0



Radial Basis Function

Solved Example

Define 4 hidden layer neurons with Gaussian RBF:

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} ; c1 = (0, 0)$$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} ; c2 = (0, 1)$$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} ; c3 = (1, 0)$$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} ; c4 = (1, 1)$$

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

For input pattern (0, 0)

Distance squared of x from $c1 = (0, 0)$

$$= (0 - 0)^2 + (0 - 0)^2 = 0$$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{\frac{0}{2}} = 1.0$$

Distance squared of x from $c2 = (0, 1)$

$$= (0 - 0)^2 + (0 - 1)^2 = 1$$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{\frac{1}{2}} = 0.6$$

Distance squared of x from $c3 = (1, 0)$

$$= (0 - 1)^2 + (0 - 0)^2 = 1$$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{\frac{1}{2}} = 0.6$$

Distance squared of x from $c4 = (1, 1)$

$$= (0 - 1)^2 + (0 - 1)^2 = 2$$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{\frac{2}{2}} = 0.4$$

$$\sum_{j=1}^m w_j H_j(x) = -0.8 \times 1.0 + 0.9 \times 0.6 + 0.9 \times 0.6 + \\ -0.8 \times 0.4 = -0.04$$

Radial Basis Function

Solved Example

Define 4 hidden layer neurons with Gaussian RBF:

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} ; c1 = (0, 0)$$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} ; c2 = (0, 1)$$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} ; c3 = (1, 0)$$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} ; c4 = (1, 1)$$

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

For input pattern (0, 1)

Distance squared of x from $c1 = (0, 0)$
 $= (0 - 0)^2 + (1 - 0)^2 = 1$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c2 = (0, 1)$
 $= (0 - 0)^2 + (1 - 1)^2 = 0$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$$

Distance squared of x from $c3 = (1, 0)$
 $= (0 - 1)^2 + (1 - 0)^2 = 2$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

Distance squared of x from $c4 = (1, 1)$
 $= (0 - 1)^2 + (1 - 1)^2 = 1$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

$$\sum_{j=1}^m w_j H_j(x) = -0.8 \times 0.6 + 0.9 \times 1.0 + 0.9 \times 0.4 + -0.8 \times 0.6 = 0.3$$

Radial Basis Function

Solved Example

Define 4 hidden layer neurons with Gaussian RBF:

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} ; c1 = (0, 0)$$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} ; c2 = (0, 1)$$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} ; c3 = (1, 0)$$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} ; c4 = (1, 1)$$

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

For input pattern (1, 0)

Distance squared of x from $c1 = (0, 0)$

$$= (1 - 0)^2 + (0 - 0)^2 = 1$$

$$H_1(x) = e^{\frac{-(x-c1)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

Distance squared of x from $c2 = (0, 1)$

$$= (1 - 0)^2 + (0 - 1)^2 = 2$$

$$H_2(x) = e^{\frac{-(x-c2)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

Distance squared of x from $c3 = (1, 0)$

$$= (1 - 1)^2 + (0 - 0)^2 = 0$$

$$H_3(x) = e^{\frac{-(x-c3)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$$

Distance squared of x from $c4 = (1, 1)$

$$= (1 - 1)^2 + (0 - 1)^2 = 1$$

$$H_4(x) = e^{\frac{-(x-c4)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

$$\sum_{j=1}^m w_j H_j(x) = -0.8 \times 0.6 + 0.9 \times 0.4 + 0.9 \times 1.0 + -0.8 \times 0.6 = 0.3$$

Radial Basis Function

Solved Example

Define 4 hidden layer neurons with Gaussian RBF:

$$H_1(x) = e^{\frac{-(x-c_1)^2}{r^2}} ; c_1 = (0, 0)$$

$$H_2(x) = e^{\frac{-(x-c_2)^2}{r^2}} ; c_2 = (0, 1)$$

$$H_3(x) = e^{\frac{-(x-c_3)^2}{r^2}} ; c_3 = (1, 0)$$

$$H_4(x) = e^{\frac{-(x-c_4)^2}{r^2}} ; c_4 = (1, 1)$$

$$d^2 = (x_2 - x_1)^2 + (y_2 - y_1)^2$$

For input pattern (1, 1)

$$\begin{aligned} \text{Distance squared of } x \text{ from } c_1 &= (0, 0) \\ &= (1-0)^2 + (1-0)^2 = 2 \end{aligned}$$

$$H_1(x) = e^{\frac{-(x-c_1)^2}{r^2}} = e^{-\frac{2}{2}} = 0.4$$

$$\begin{aligned} \text{Distance squared of } x \text{ from } c_2 &= (0, 1) \\ &= (1-0)^2 + (1-1)^2 = 1 \end{aligned}$$

$$H_2(x) = e^{\frac{-(x-c_2)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

$$\begin{aligned} \text{Distance squared of } x \text{ from } c_3 &= (1, 0) \\ &= (1-1)^2 + (1-0)^2 = 1 \end{aligned}$$

$$H_3(x) = e^{\frac{-(x-c_3)^2}{r^2}} = e^{-\frac{1}{2}} = 0.6$$

$$\begin{aligned} \text{Distance squared of } x \text{ from } c_4 &= (1, 1) \\ &= (1-1)^2 + (1-1)^2 = 0 \end{aligned}$$

$$H_4(x) = e^{\frac{-(x-c_4)^2}{r^2}} = e^{-\frac{0}{2}} = 1.0$$

$$\begin{aligned} \sum_{j=1}^m w_j H_j(x) &= -0.8 \times 0.4 + 0.9 \times 0.6 + 0.9 \times 0.6 + \\ &-0.8 \times 1.0 = -0.04 \end{aligned}$$

Radial Basis Function

Solved Example

Forward Phase Calculation

Input		$H_1(x)$	$H_2(x)$	$H_3(x)$	$H_4(x)$	$\sum_{j=1}^m w_j H_j(x)$	Output
0	0	1.0	0.6	0.6	0.4	-0.04	0
0	1	0.6	1.0	0.4	0.6	0.3	1
1	0	0.6	0.4	1.0	0.6	0.3	1
1	1	0.4	0.6	0.6	1.0	-0.04	0
		$w_1 = -0.8$	$w_2 = 0.9$	$w_3 = 0.9$	$w_4 = -0.8$		

INTERPOLATION AND BASIS FUNCTIONS

- One of the problems is function approximation: given some data, find a function that goes through the data without over-fitting to the noise, so that values between the known data points can be inferred or interpolated.
- RBF network solves this problem by each of the basis functions making a contribution to the output whenever the input is within its receptive field. So several RBF nodes will probably respond for each input.

- the receptive fields are not allowed to overlap, and space them out so that they just meet up with each other.
- Gaussian part is not needed as that decides how much each one matches now, either.
- If the data point is within the receptive field of this function then listen only to this function, otherwise ignore it and listen to some other function.
- If each function just returns the average value within its patch, then for one-dimensional data we get a histogram output, Fig 5.5.
- Extend this a bit further so that the lines are not horizontal, but instead reflect the first derivative of the curve at that point, Fig 5.6.
- The output to be continuous, so that the line within the first bin meets up with the line in the second bin at the boundary, so add the extra constraint that the lines have to meet up as well. Fig 5.7

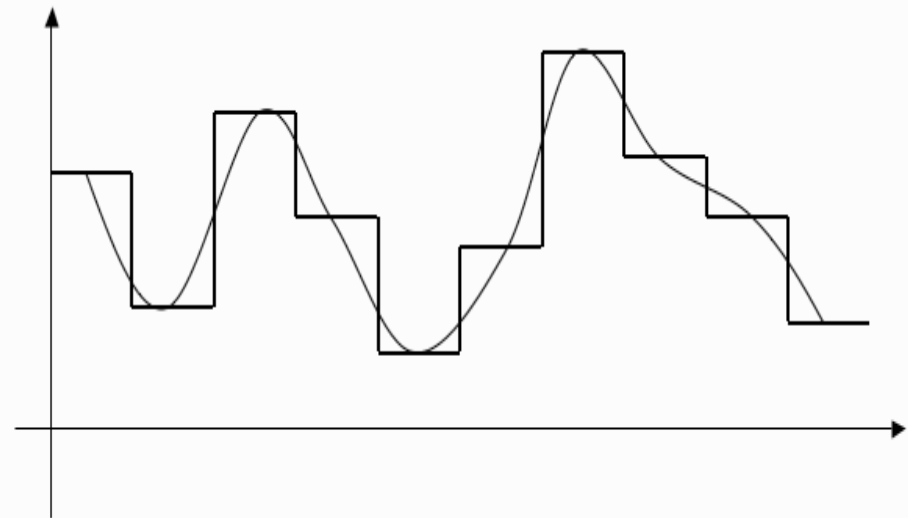
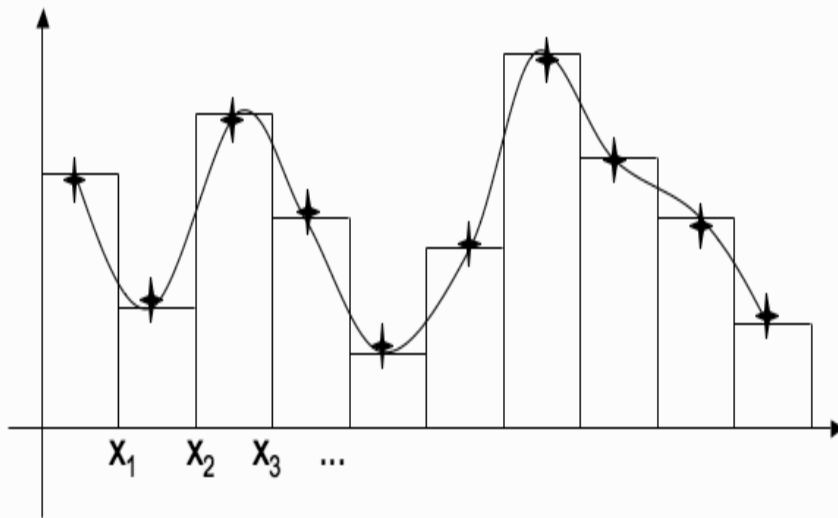
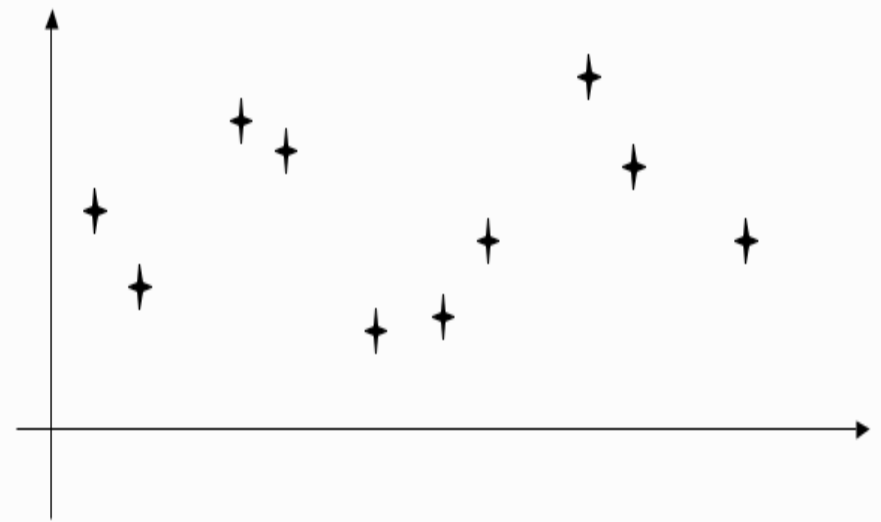
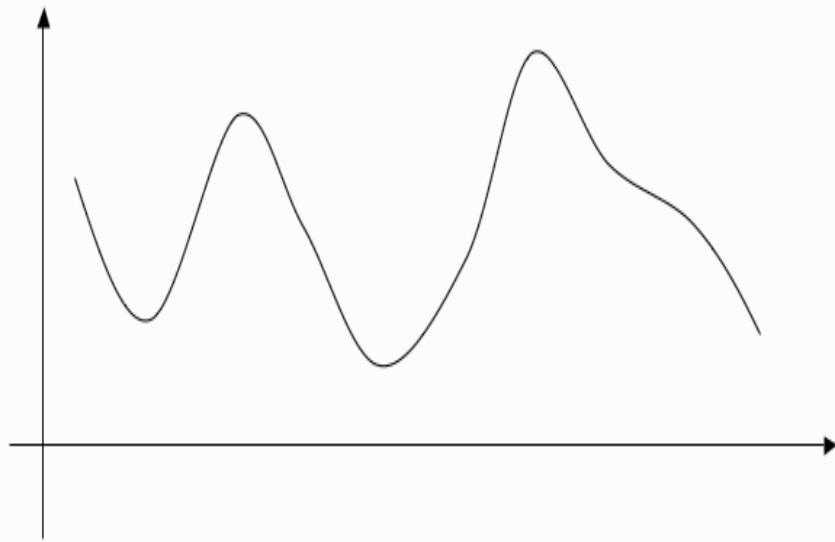


FIG 5.5 *Top: Curve showing a function. Second: A set of data points from the curve. Third: Putting a straight horizontal line through each point creates a histogram that describes an approximation to the curve. Bottom: That approximation.*

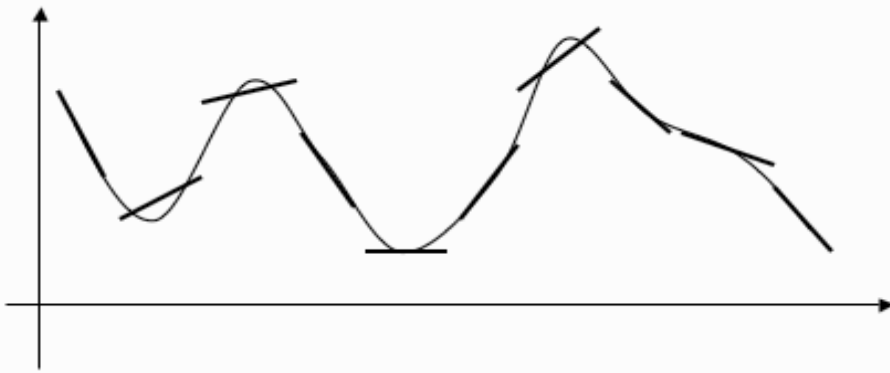


FIGURE 5.6 Representing the points by straight lines that aren't necessarily horizontal (so that their first derivative matches at the point) gives a better approximation

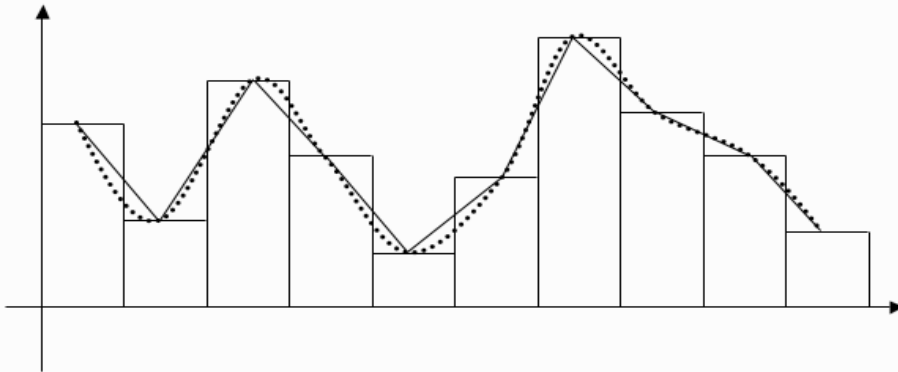


FIG5.7 Making the straight lines meet so that the function is continuous gives a better approximation.

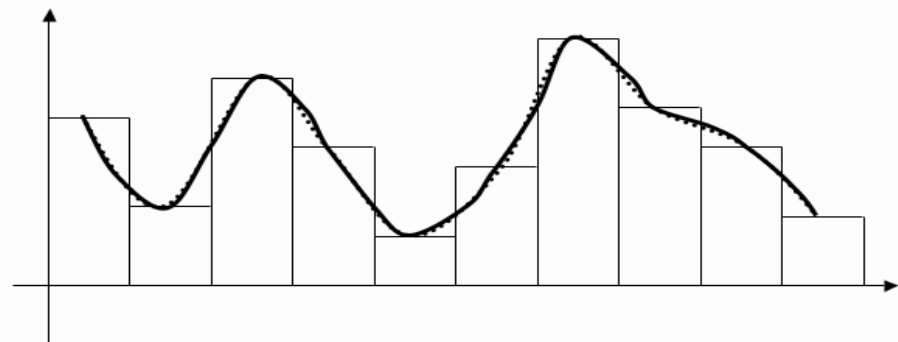


FIG 5.8 Using cubic functions to connect the points gives an even better approximation, and the curve is also continuous at the points where the sections join up (known as knot points).

- These functions are known as splines, and the most common one to use is the cubic spline.

Bases and Basis Expansion

- Radial basis functions and several other machine learning algorithms can be written in this form:

$$f(\mathbf{x}) = \sum_{i=1}^n \alpha_i \Phi_i(\mathbf{x}), \quad (5.3)$$

- where $\Phi_i(\mathbf{x})$ is some function of the input value $\mathbf{x}$ and the α_i are the parameters we can solve for in order to make the model fit the data.

5.3.2 The Cubic Spline

We can carry on adding extra powers of x , but it turns out that the cubic spline is generally sufficient. This has four basic basis functions ($\Phi_1(x) = 1, \Phi_2(x) = x, \Phi_3(x) = x^2, \Phi_4(x) = x^3$), and then as many extras as there are knotpoints, each of the form $\Phi_{4+i}(x) = (x - x_i)_+^3$. This function constrains the function itself and also its first two derivatives to meet at each knotpoint. Notice that while the Φ s are not linear, we are simply adding up a weighted sum of them, and so the model is linear in them. We can then produce curves like Figure 5.8, which represent the data very well.

5.3.3 Fitting the Spline to the Data

Having defined the functions, we need to work out how to choose the α_i in order to make the model fit the data. We will continue to define the sum-of-squares error and to minimise that, which is known in the statistical literature as least-squares fitting, and will be described in more detail in Section 9.2. The important point is that everything is linear in the basis functions, so computing the least-squares fit is a linear problem. As with the MLP, the error that we are trying to minimise is:

$$E(y, f(x)) = \sum_{i=1}^N (y_i - f(x_i))^2. \quad (5.4)$$

5.3.4 Smoothing Splines

The way that we constructed the splines in Section 5.3 was to insist that they went through each knotpoint exactly. This was a good way to describe our constraints, but it is not necessarily realistic: almost all of the data that we ever see will be noisy, and insisting that the data goes through the knotpoints therefore overfits: imagine that the line in Figure 5.9 went through each datapoint. As we try to make the spline model match the data more and more accurately, we will add further knotpoints, which leads to further overfitting. We can deal with this by using *regularisation*. This is a very important idea in optimisation. In essence, it means adding an extra constraint that makes the problem simpler to solve by providing some way to choose from amongst the set of possible solutions.

The most common regulariser that is used for splines is to make the spline model as ‘smooth’ as possible, where the smoothness is measured by computing the second derivative of the curve at each point, squaring it so that it is always positive, and integrating it along the curve. In this way, a straight line is perfectly smooth, but probably won’t be a good match for the data, so we introduce a parameter λ that describes the tradeoff between the two parts. We regain the interpolating spline of Section 5.3 for $\lambda = 0$, whereas for $\lambda \rightarrow \infty$ we get the least-squares straight line. This type of spline is known as a **smoothing spline**. The cubic smoothing spline is often used. While there are automated methods of choosing λ , it is more normal to use cross-validation to find a value that seems to work well. The form of the optimisation is now:

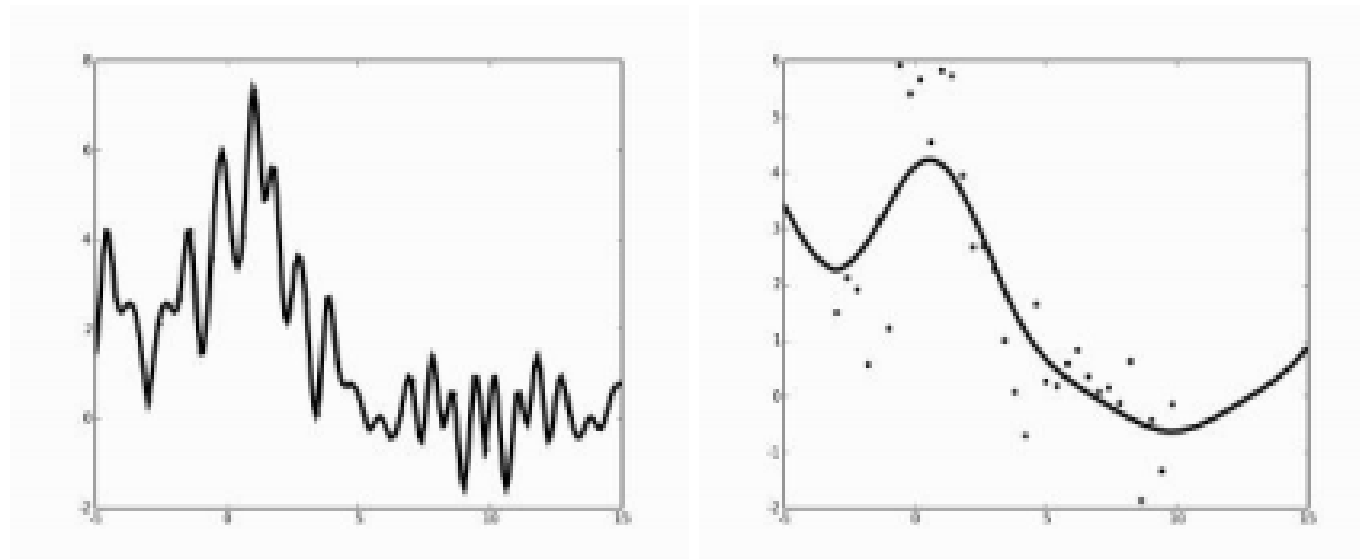


FIGURE 5.10 B-spline fitting of the data shown in Figure 5.9 with *left*: $\lambda = 0$ and *right*: $\lambda = 100$.

$$E(y, f(x), \lambda) = \sum_{i=1}^N (y_i - f(x_i))^2 + \lambda \int \left(\frac{d^2 f}{dt^2} \right)^2 dt. \quad (5.5)$$

Support Vector Machines

- Kernel functions are introduced rather than bases
- Perceptron was rather limited in that it could only identify straight line classifiers, that is, it could only separate out groups of data if it was possible to draw a straight line (hyperplane in higher dimensions) between them.
- This meant that it could not learn to distinguish between the two truth classes of the 2D XOR function.
- Possible to modify the problem so that the Perceptron could solve the problem by changing the data so that it used more dimensions than the original data.
 - modify the data by changing its representation
 - introduce kernel functions rather than bases
- The problem is how to work out which dimensions to use, and that is what kernel methods, which is the class of algorithms.

- Support Vector Machine (SVM) , which is one of the most popular algorithms in modern machine learning.
- It is a supervised machine learning algorithm.
- Use for classification and regression, Mostly used for classification.
- It was introduced by Vapnik in 1992 and has taken off radically since then, principally because it often (but not always) provides very impressive classification performance on reasonably sized datasets.
- SVMs do not work well on extremely large datasets, since the computations don't scale well with the number of training examples, and so become computationally very expensive.
- Two types i) Linear SVM ii) Non-linear SVM

- In this algorithm we plot data item as a point in n -dimensional space (n is the no. of features) with the value of each feature being the value of particular coordinate

Hyperplane: A hyperplane is defined as an $n-1$ dimensional euclidean space that separates an n -dimensional euclidean space into two disconnected parts or classes

- “**Optimal Hyperplane**” is one that maximizes the margin between the two classes (which means the distance of the hyperplane is equal from the both the nearest data points of the two classes



- It is this distinction that enables the SVM algorithm to be derived, so that is where we will start.

OPTIMAL SEPARATION

- Figure 8.1 shows a simple classification problem with three different possible linear classification lines.
- All three of the lines that are drawn separate out the two classes, so in some sense they are 'correct', and the Perceptron would stop its training if it reached any one of them.

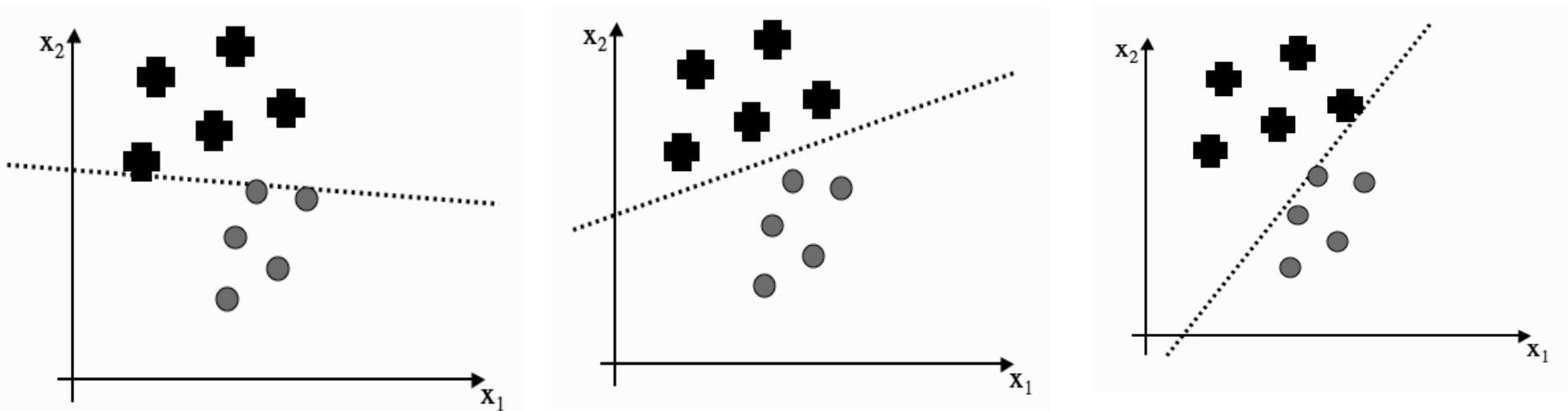


FIG 8.1 Three different classification lines. Is there any reason why one is better than the others?

- However, if you had to pick one of the lines to act as the classifier for a set of test data, most of you would pick the middle picture.
- It's probably hard to describe exactly why you would do this, but somehow we prefer a line that runs through the middle of the separation between the data points from the two classes, staying approximately equidistant from the data in both classes.

THE MARGIN AND SUPPORT VECTORS

- Measure the distance that we have to travel away from the line (in a direction perpendicular to the line) before we hit a data point.
- Imagine that we put a 'no-man's land' around the line (shown in Fig 8.2), so that any point that lies within that region is declared to be too close to the line to be accurately classified.
- This region is symmetric about the line, it forms a cylinder about the line in 3D, and a hyper-cylinder in higher dimensions.
- How large could we make the radius of this cylinder until we started to put points into a no-man's land, don't know which class they are from? This largest radius is known as the margin, labelled M .

- The classifier in the middle of Figure 8.1 has the largest margin of the three.
- It has the imaginative name of the maximum margin (linear) classifier.
- The data points in each class that lie closest to the classification line have a name as well. They are called **support vectors**.
- Using the argument that the best classifier is the one that goes through the middle of no-man's land, now make two arguments: first that the margin should be as large as possible, and second that the support vectors are the most useful data points.
- This leads to an interesting feature of these algorithms: after training, throw away all of the data except for the support vectors, and use them for classification, which is a useful saving in data storage.
- A measurement that we can use to find the optimal decision boundary

- A weight vector (a vector, not a matrix, since there is only one output) and an input vector x .
- The output used is $y = w \cdot x + b$, with b being the contribution from the bias weight.
- Use the classifier line by saying that any x value that gives a positive value for $w \cdot x + b$ is above the line, and so is an example of the '+' class, while any x that gives a negative value is in the '-' class.
- In our new version want to include our no-man's land.
- whether the value of $w \cdot x + b$ is positive or negative, check for absolute value is less than our margin M , which would put it inside the grey box in Fig 8.2.

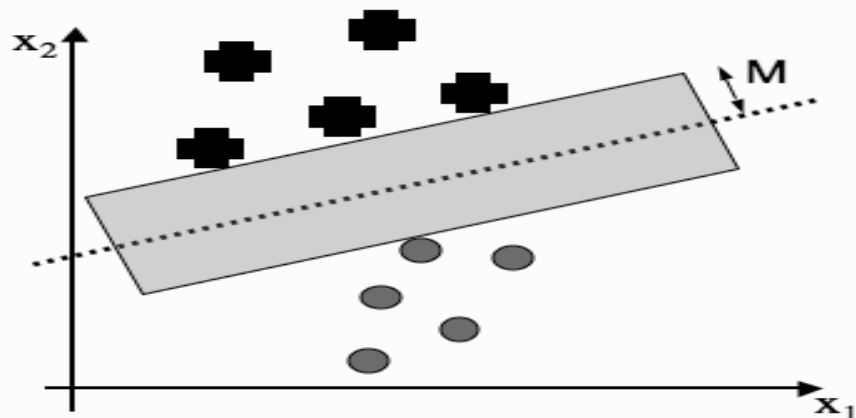
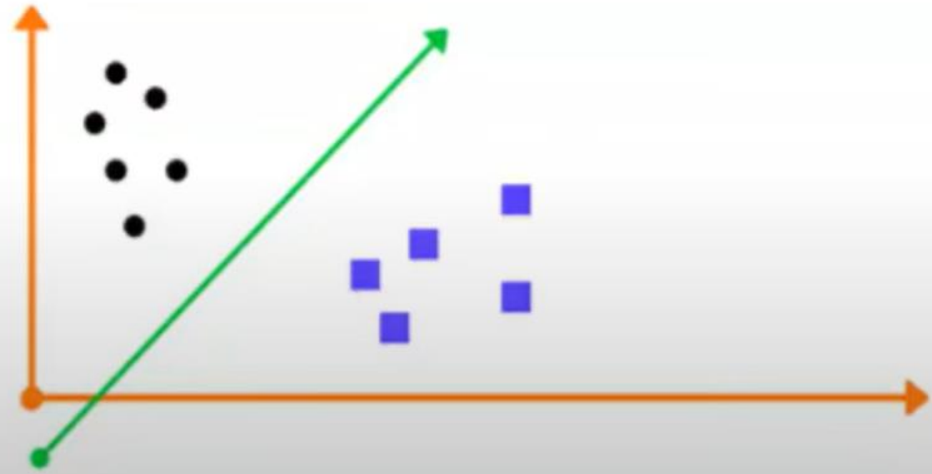


FIGURE 8.2 The margin is the largest region we can put that separates the classes without there being any points inside, where the box is made from two lines that are parallel to the decision boundary.

- $w \cdot x$ is the inner or scalar product, $w \cdot x = \sum_i w_i x_i$
- *This can also be written as $w^T x$, means treat the vectors as degenerate matrices and use the normal matrix multiplication rules.*
- This notation will turn out to be simpler, and so will be used from here on.
- For a given margin value M we can say that any point x where $w^T x + b \geq M$ is a plus, and any point where $w^T x + b \leq -M$ is a circle. The actual separating hyper plane is specified by $w^T x + b = 0$.
- *pick a point x^+ that lies on the '+' class boundary line, so that*
- $w^T x^+ = M$
- This is a support vector. If we want to find the closest point that lies on the boundary line for the '-' class, then we travel perpendicular to the '+' boundary line until we hit the '-' boundary line.
- The point that we hit is the closest point, and we'll call it x^- .
- Requirements as: minimise $\frac{1}{2} w^T w$ (where the half is there for convenience as in so many other cases)

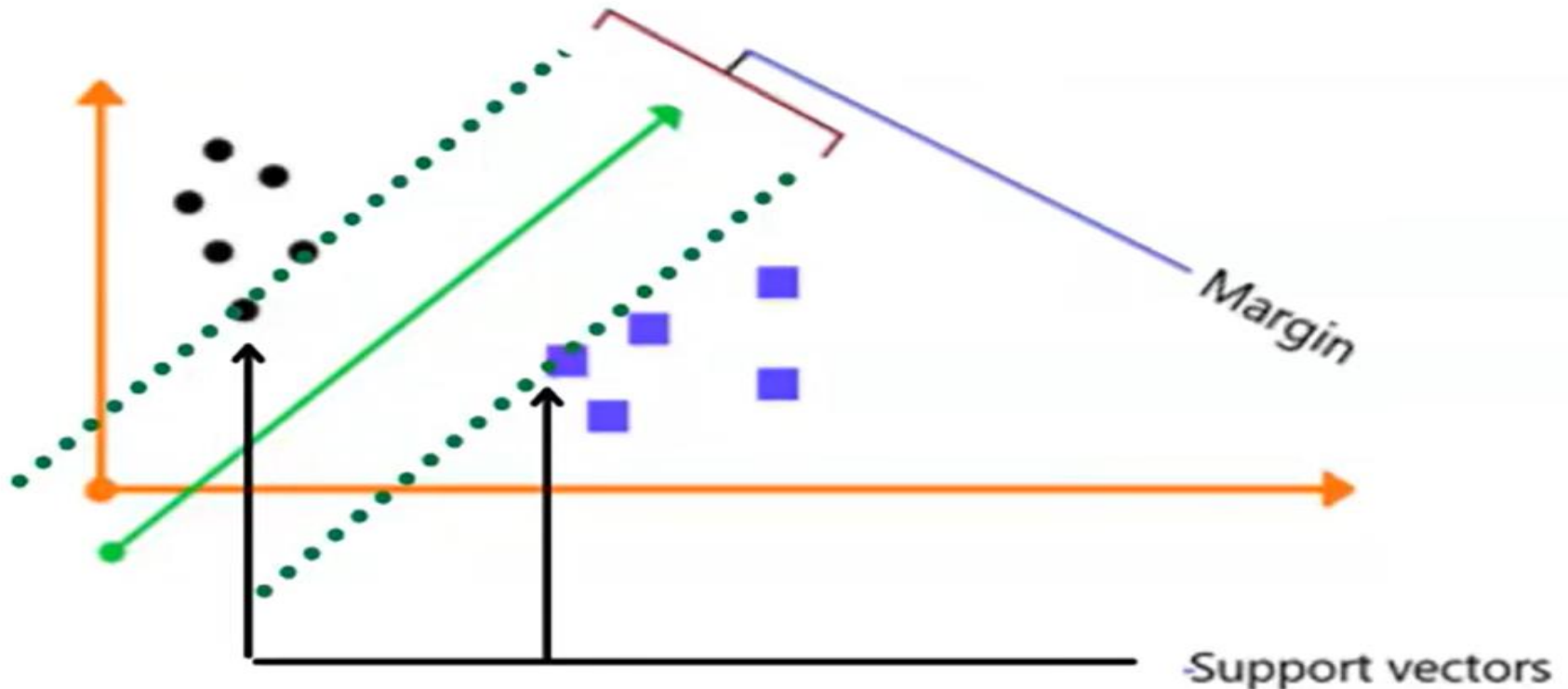
This is exactly how SVM works, it draws a decision boundary which is a hyperplane between any two classes in order to separate them or classify them.



- However, if you had to pick one of the lines to act as the classifier for a set of test data, most of you would pick the middle picture.
- It's probably hard to describe exactly why you would do this, but somehow we prefer a line that runs through the middle of the separation between the data points from the two classes, staying approximately equidistant from the data in both classes.

THE MARGIN AND SUPPORT VECTORS :

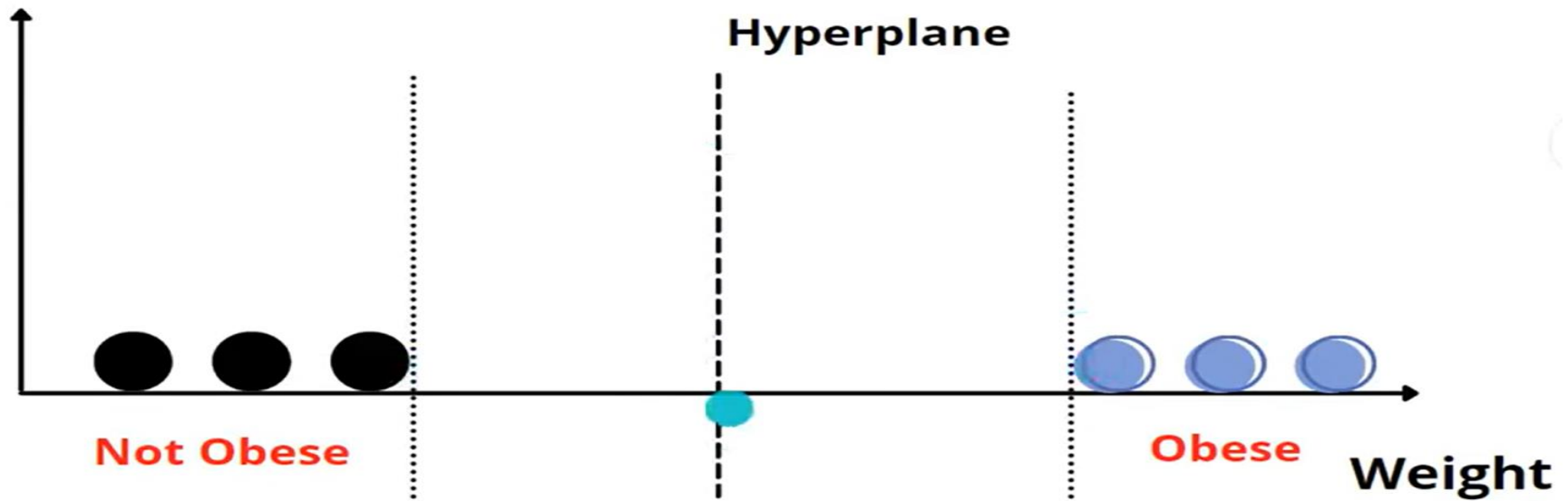
- The basic principles behind the SVM is to draw a hyperplane that best separates the two classes. so we start drawing a random hyperplane and check distance between the hyperplane and closest data points from each class
- Measure the distance that we have to travel away from the line (in a direction perpendicular to the line) before we hit a data point

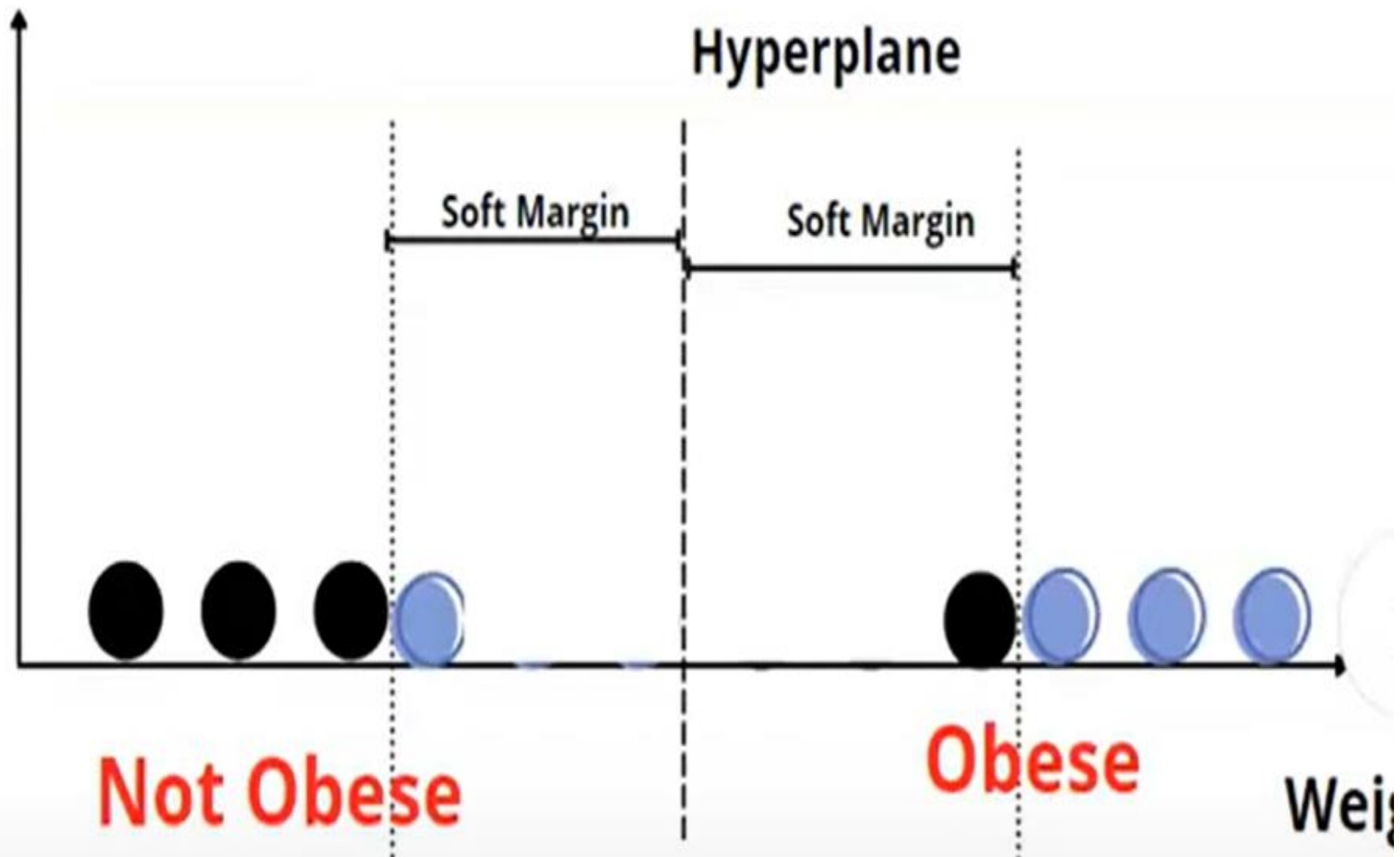


- The data points in each class that lie closest to the classification line have a name as well. They are called **support vectors**. (or)
- Closest on your data points to the hyperplane are known as support vectors and that's where the name comes from support vector machine so basically the hyperplane is drawn based on these support vectors
- Normally the hyperplane which has the maximum distance from the support vectors is the most optimum hyperplane and this distance between the hyperplane and the support vector is known as **margin**.
- A measurement that we can use to find the optimal decision boundary

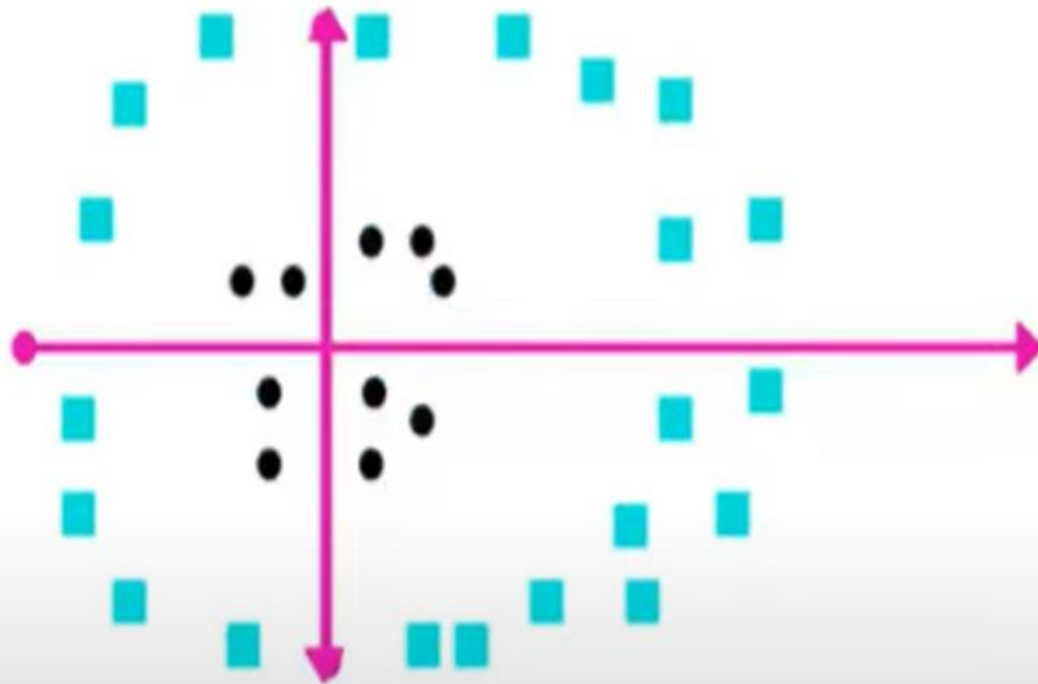
Support vector classifier:

- Support vector classifier is a soft margin classifier
- We will allow some observations to be incorrectly classified or to be on the wrong side of the margin
- Is a linear classifier, it cant classify non-linear data





Support Vector Machine:

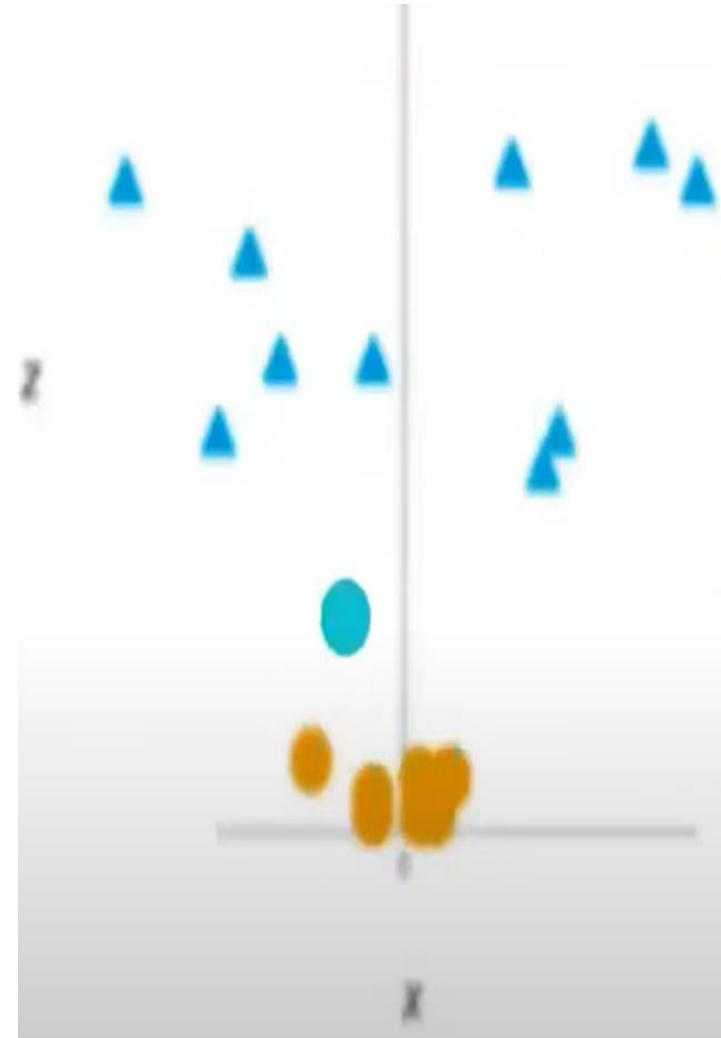


how do I
separate this
data?

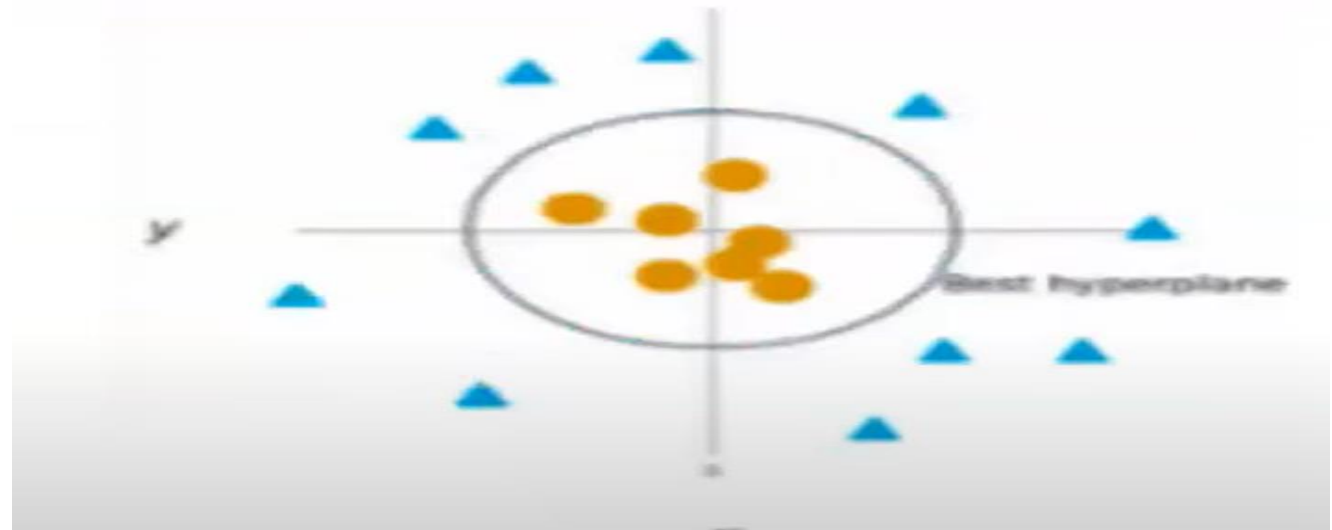
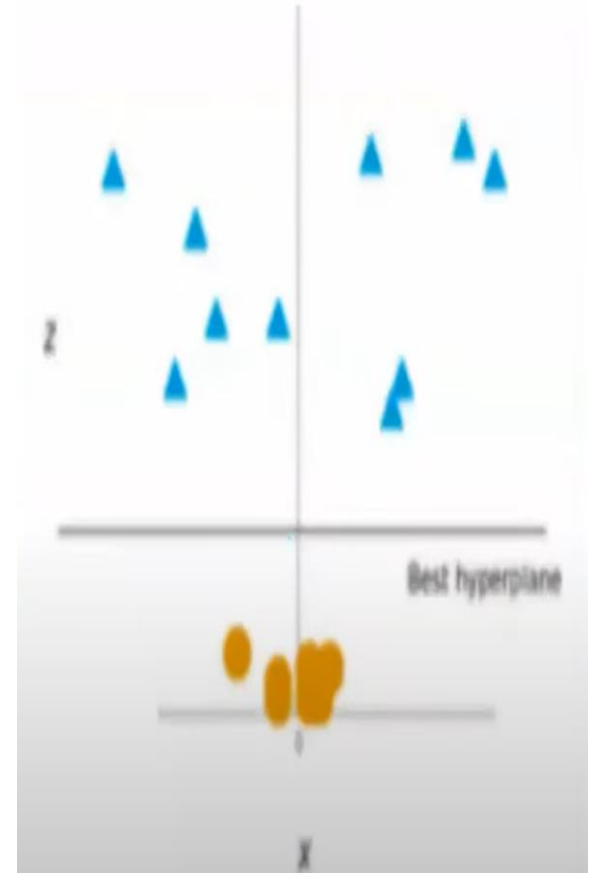


Non-linear SVM is used when the data can't be separated using a straight line.

- Support Vector machine(SVM) is an extension of support vector classifier which uses kernels to create non linear boundaries.
 - Linear kernel
 - polynomial kernel
 - Radial basis function of RBF kernel
 - Sigmoid kernel
 - Gaussian kernel
- In the above case we cant draw a linear boundary, we will now add a third dimension using kernel
- we create a new z dimensional, and we rule that it be calculated a certain way that is convenient for us
 $Z = x^2 + y^2$ (equation for a circle)



- This will give us 3-Dimensional space. From a different perspective, the data is now in two linearly separated groups.
- All values for z would be always positive because z is the squared sum of both x and y .
- Now we can map back to 2-Dimensions. our decision boundary is a circumference which separates both tags using SVM. we got circle as a hyperplane.



A Constrained Optimisation Problem

- Make the target answers for our two classes be ± 1 , rather than 0 and 1.
- $t_i \times y_i$, *target multiplied by* the output, and this will be positive if the two are the same and negative otherwise.
- write down the equation of the straight line again, which is how we computed y , require that $t_i(w^T x + b) \geq 1$
- Minimise $\frac{1}{2}w^T w$ subject to $t_i(w^T x + b) \geq 1$ for all $i = 1, \dots, n$. (8.1)
- Quadratic programming, takes advantage of the fact that the problem described is quadratic and therefore convex, and has linear constraints.
- A convex problem is one, take any two points on the line and join them with a straight line, then every point on the line will be above the curve.
- Fig 8.4 shows an example of a convex and a non-convex function.

- Convex functions have a unique minimum, which is fairly easy to see in one dimension, and remains true in any number of dimensions.

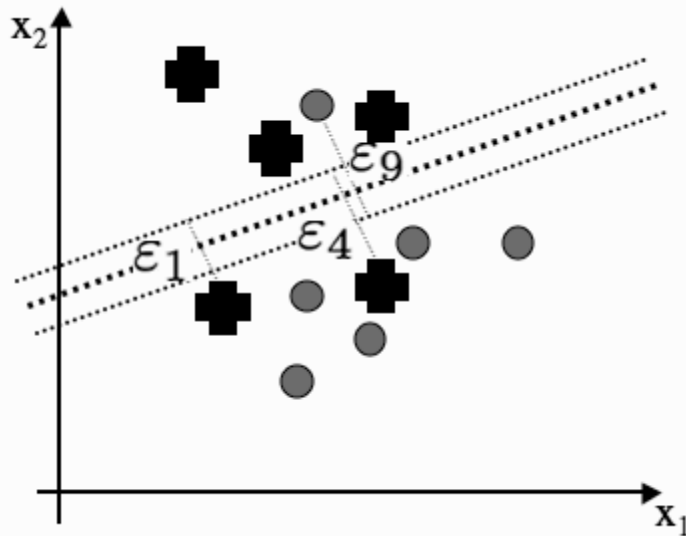


FIG8.3 If the classifier makes some errors, then the distance by which the points are over the border should be used to weight each error in order to decide how bad the classifier is.

- Since the problem is quadratic, there is a unique optimum.
- When we find that optimal solution, the Karush–Kuhn–Tucker (KKT) conditions will be satisfied.
- These are (for all values of i from 1 to n , and where the $*$ denotes the optimal value of each parameter):

$$\lambda_i^*(1 - t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*)) = 0 \quad (8.2)$$

$$1 - t_i(\mathbf{w}^{*T} \mathbf{x}_i + b^*) \leq 0 \quad (8.3)$$

$$\lambda_i^* \geq 0, \quad (8.4)$$

- where the λ_i are *positive values known as Lagrange multipliers, which are a standard approach* to solving equations with equality constraints.
- The first of these conditions tells us that if $\lambda_i = 0$ then $(1 - t_i(\mathbf{w}^T \mathbf{x}_i + b)) = 0$.
- This is only true for the support vectors (the SVMs provide a sparse representation of the data), and so we only have to consider them, and can ignore the rest.
- The support vectors are those vectors in the active set of constraints.
- For the support vectors the constraints are equalities instead of inequalities.
- We can therefore solve the Lagrangian function:

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\lambda}) = \frac{1}{2} \mathbf{w}^T \mathbf{w} + \sum_{i=1}^n \lambda_i (1 - t_i (\mathbf{w}^T \mathbf{x}_i + b)), \quad (8.5)$$

We differentiate this function with respect to the elements of $\mathbf{w}$ and b :

$$\nabla_{\mathbf{w}} \mathcal{L} = \mathbf{w} - \sum_{i=1}^n \lambda_i t_i \mathbf{x}_i, \quad (8.6)$$

and

$$\frac{\partial \mathcal{L}}{\partial b} = - \sum_{i=1}^n \lambda_i t_i. \quad (8.7)$$

If we set the derivatives to be equal to zero, so that we find the saddle points (maxima) of the function, we see that:

$$\mathbf{w}^* = \sum_{i=1}^n \lambda_i t_i \mathbf{x}_i, \quad \sum_{i=1}^n \lambda_i t_i = 0. \quad (8.8)$$

We can substitute these expressions at the optimal values of $\mathbf{w}$ and b into Equation (8.5) and, after a little bit of rearranging, we get (where $\boldsymbol{\lambda}$ is the vector of the λ_i):

$$\mathcal{L}(\mathbf{w}^*, b^*, \boldsymbol{\lambda}) = \sum_{i=1}^n \lambda_i - \sum_{i=1}^n \lambda_i t_i - \frac{1}{2} \sum_{i=1}^n \sum_{j=1}^n \lambda_i \lambda_j t_i t_j \mathbf{x}_i^T \mathbf{x}_j, \quad (8.9)$$

and we can notice that using the derivative with respect to b we can treat the middle term as 0. This equation is known as the dual problem, and the aim is to maximise it with respect to the λ_i variables. The constraints are that $\lambda_i \geq 0$ for all i , and $\sum_{i=1}^n \lambda_i t_i = 0$.

Equation (8.8) gives us an expression for $\mathbf{w}^*$, but we also want to know what b^* is. We know that for a support vector $t_i(\mathbf{w}^T \mathbf{x}_i + b) = 1$, and we can substitute the expression for

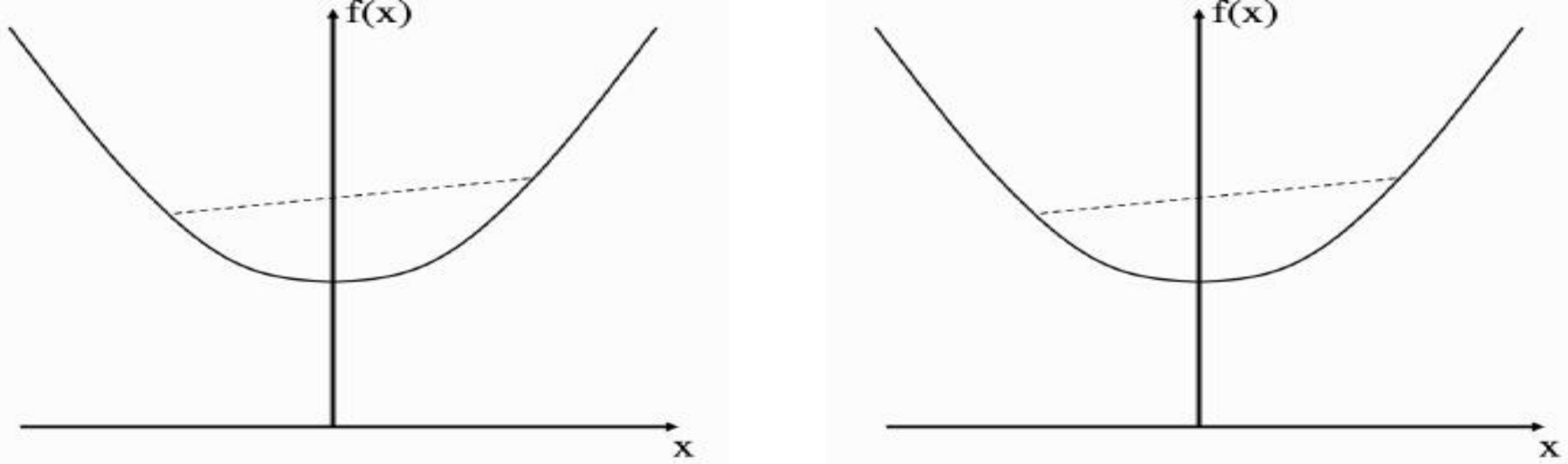


FIG8.4 A function is convex if every straight line that links two points on the curve does not intersect the curve anywhere else. The function on the left is convex, but the one on the right is not, as the dashed line shows.

- W^* into there and substitute in the (x, t) of *one of the support vectors*.
- *However, in case of errors this is not very stable, and so it is better to average it over the whole set of N_s support vectors:*

$$b^* = \frac{1}{N_s} \sum_{\text{support vectors } j} \left(t_j - \sum_{i=1}^n \lambda_i t_i \mathbf{x}_i^T \mathbf{x}_j \right). \quad (8.10)$$

We can also use Equation (8.8) to see how to make a prediction, since for a new point $\mathbf{z}$:

$$\mathbf{w}^{*T} \mathbf{z} + b^* = \left(\sum_{i=1}^n \lambda_i t_i \mathbf{x}_i \right)^T \mathbf{z} + b^*. \quad (8.11)$$

This means that to classify a new point, we just need to compute the inner product between the new datapoint and the support vectors.

SLACK VARIABLES FOR NON-LINEARLY SEPARABLE PROBLEMS

- Everything that we have done so far has assumed that the data set is linearly separable.
- If we have a non-linearly separable dataset, then cannot satisfy the constraints for all of the data points.
- The solution is to introduce some slack variables $\eta_i \geq 0$ so that the constraints become $t_i(\mathbf{w}^T \mathbf{x}_i + b) \geq 1 - \eta_i$. For inputs that are correct, we set $\eta_i = 0$.

- These slack variables are telling us that, when comparing classifiers, we should consider the case where one classifier makes a mistake by putting a point just on the wrong side of the line, and another puts the same point a long way onto the wrong side of the line.
- minimise $w^T w + C \times (\text{distance of misclassified points from the correct boundary line})$.
- Here, C is a tradeoff parameter that decides how much weight to put onto each of the two criteria: small C means we prize a large margin over a few errors, while large C means the opposite.
- This transforms the problem into a soft-margin classifier, since we are allowing for a few mistakes. Minimize

$$L(w, \epsilon) = w^T w + C \sum_{i=1}^n \epsilon_i. \quad (8.12)$$

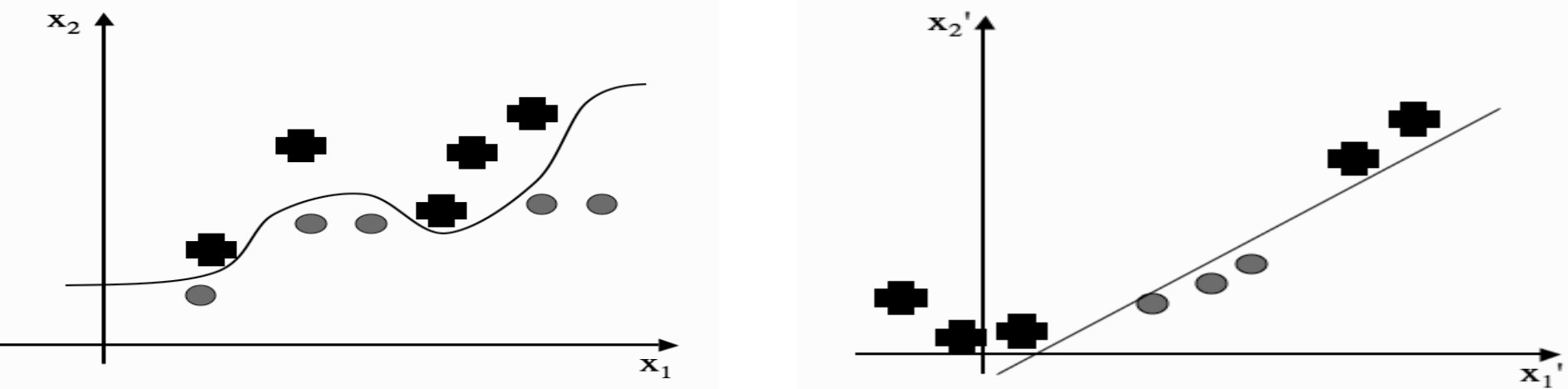


FIGURE 8.5 By modifying the features we hope to find spaces where the data are linearly separable.

KERNELS

- Introduce new functions (ϕ) of our input features. Replace x_i by $\phi(x_i)$ (and z by $\phi(z)$) and get a prediction quite easily.

$$w^T x + b = \left(\sum_{i=1}^n \lambda_i t_i \phi(x_i) \right)^T \phi(z) + b. \quad (8.16)$$

CHOOSING KERNELS

- Any symmetric function that is positive definite (meaning that it enforces positivity on the integral of arbitrary functions) can be used as a kernel.
- This is a result of Mercer's theorem, which also says that it is possible to convolve kernels together and the result will be another kernel.

- polynomials up to some degree s in the elements x_k of the input vector (e.g., x_3^3 or $x_1 \times x_4$) with kernel:

$$\mathbf{K}(\mathbf{x}, \mathbf{y}) = (1 + \mathbf{x}^T \mathbf{y})^s \quad (8.19)$$

For $s = 1$ this gives a linear kernel

- sigmoid functions of the x_k s with parameters κ and δ , and kernel:

$$\mathbf{K}(\mathbf{x}, \mathbf{y}) = \tanh(\kappa \mathbf{x}^T \mathbf{y} - \delta) \quad (8.20)$$

- radial basis function expansions of the x_k s with parameter σ and kernel:

$$\mathbf{K}(\mathbf{x}, \mathbf{y}) = \exp \left(-(\mathbf{x} - \mathbf{y})^2 / 2\sigma^2 \right) \quad (8.21)$$

- Choosing which kernel to use and the parameters in these kernels is a tricky problem.
- While there is some theory based on something known as the Vapnik–Chernik dimension that can be applied, most people just experiment with different values and find one that works, using a validation set as we did for the MLP.

THE SUPPORT VECTOR MACHINE ALGORITHM

- Use cvxopt, which is a convex optimisation package that includes a wrapper for Python.
- There is a link to the relevant website on the book webpage. cvxopt has a nice and clean interface so we can use this to do the computational heavy lifting for an implementation of the SVM.

- Initialisation

- for the specified kernel, and kernel parameters, compute the kernel of distances between the datapoints
 - * the main work here is the computation $\mathbf{K} = \mathbf{X}\mathbf{X}^T$
 - * for the linear kernel, return $\mathbf{K}$, for the polynomial of degree d return $\frac{1}{\sigma} \mathbf{K}^d$
 - * for the RBF kernel, compute $\mathbf{K} = \exp(-(\mathbf{x} - \mathbf{x}')^2/2\sigma^2)$

- Training

- assemble the constraint set as matrices to solve:

$$\min_{\mathbf{x}} \frac{1}{2} \mathbf{x}^T \mathbf{t}_i \mathbf{t}_j \mathbf{K} \mathbf{x} + \mathbf{q}^T \mathbf{x} \text{ subject to } \mathbf{G} \mathbf{x} \leq \mathbf{h}, \mathbf{A} \mathbf{x} = \mathbf{b}$$

- pass these matrices to the solver
- identify the support vectors as those that are within some specified distance of the closest point and dispose of the rest of the training data
- compute b^* using equation (8.10)

- Classification

- for the given test data $\mathbf{z}$, use the support vectors to classify the data for the relevant kernel using:
 - * compute the inner product of the test data and the support vectors
 - * perform the classification as $\sum_{i=1}^n \lambda_i t_i \mathbf{K}(\mathbf{x}_i, \mathbf{z}) + b^*$, returning either the label (hard classification) or the value (soft classification)

Case Study:

- This is iris flowers, with petal , sepal width and height, based on these four features we determine whether it is setosa or versicolor



